DeepL
Subscribe to DeepL Pro to translate larger documents.
Visit www.DeepL.com/pro for more information.

CHƯƠNG 4

Huấn luyện mô hình

Cho đến nay, chúng ta đã chủ yếu xem các mô hình học máy và các thuật toán huấn luyện của chúng như những "hộp đen". Nếu bạn đã thực hiện một số bài tập trong các chương trước, có lẽ bạn đã ngạc nhiên về những gì mình có thể làm được mà không cần biết gì về cơ chế bên trong: bạn đã tối ưu hóa một hệ thống hồi quy, cải thiện một bộ phân loại hình ảnh chữ số, và thậm chí xây dựng một bộ phân loại thư rác từ đầu, tất cả đều mà không cần biết chúng thực sự hoạt động như thế nào. Thực tế, trong nhiều tình huống, bạn không thực sự cần biết chi tiết về cách triển khai.

Tuy nhiên, việc hiểu rõ cách thức hoạt động của các mô hình có thể giúp bạn nhanh chóng xác định mô hình phù hợp, thuật toán huấn luyện đúng đắn và bộ siêu tham số tối ưu cho tác vụ của mình. Hiểu rõ cơ chế bên trong cũng sẽ giúp bạn gỡ lỗi và phân tích lỗi hiệu quả hơn. Cuối cùng, hầu hết các chủ đề được thảo luận trong chương này sẽ là nền tảng thiết yếu để hiểu, xây dựng và huấn luyện mạng nơ-ron (sẽ được đề cập trong Phần II của cuốn sách này).

Trong chương này, chúng ta sẽ bắt đầu bằng việc xem xét mô hình hồi quy tuyến tính, một trong những mô hình đơn giản nhất hiện có. Chúng ta sẽ thảo luận về hai cách huấn luyện rất khác nhau:

- Sử dụng phương trình "dạng đóng"[1] để tính toán trực tiếp các tham số mô hình phù hợp nhất với tập huấn luyện (tức là các tham số mô hình làm tối thiểu hàm chi phí trên tập huấn luyện).
- Sử dụng phương pháp tối ưu hóa lặp gọi là gradient descent (GD), từ từ điều chỉnh các tham số mô hình để tối thiểu hóa hàm chi phí trên tập huấn luyện, cuối cùng hội tụ đến cùng một bộ tham số như phương pháp đầu tiên. Chúng ta sẽ xem xét một số biến thể của gradient descent mà chúng ta sẽ sử dụng lại và

1 Một phương trình dạng đóng chỉ bao gồm một số hữu hạn các hằng số, biến và phép toán tiêu chuẩn: ví dụ, $a = \sin(b - c)$. Không có tổng vô hạn, không có giới hạn, không có tích phân, v.v.

một lần nữa khi chúng ta nghiên cứu mạng nơ-ron trong Phần II: GD theo lô, GD theo mini-lô và GD ngẫu nhiên.

Tiếp theo, chúng ta sẽ xem xét hồi quy đa thức, một mô hình phức tạp hơn có thể phù hợp với các tập dữ liệu phi tuyến. Vì mô hình này có nhiều tham số hơn hồi quy tuyến tính, nên nó dễ bị hiện tượng quá khớp với dữ liệu huấn luyện hơn. Chúng ta sẽ tìm hiểu cách phát hiện liệu điều này có xảy ra hay không bằng cách sử dụng các đường cong học tập, và sau đó chúng ta sẽ xem xét một số kỹ thuật thường quy hóa có thể giảm thiểu rủi ro quá khớp với tập huấn luyện.

Cuối cùng, chúng ta sẽ xem xét thêm hai mô hình thường được sử dụng cho các tác vụ phân loại: hồi quy logistic và hồi quy softmax.



Chương này sẽ có khá nhiều phương trình toán học sử dụng các khái niệm cơ bản của đại số tuyến tính và giải tích. Để hiểu các phương trình này, bạn cần nắm vững các khái niệm về vectơ và ma trận — cách chuyển vị, nhân và đảo ngược chúng — cũng như đạo hàm riêng. Nếu các khái niệm này còn xa lạ, vui lòng ôn lại các notebook Jupyter giới thiệu về đại số tuyến tính và giải tích được cung cấp trong tài liệu bổ sung trực tuyến. Nếu bạn thực sự "dị ứng" với toán học, bạn có thể bỏ qua các phương trình; nội dung văn bản vẫn sẽ giúp bạn nắm bắt hầu hết các khái niệm. Tuy nhiên, việc học các khái niệm toán học chính thức là cực kỳ hữu ích, vì nó sẽ giúp bạn đọc các bài báo về học máy. Mặc dù ban đầu có vẻ đáng sợ, nhưng thực ra nó không quá khó, và chương này bao gồm mã nguồn sẽ giúp bạn hiểu rõ các phương trình.

Hồi quy tuyến tính

Trong Chương 1, chúng ta đã xem xét một mô hình tuyến tính đơn giản về mức độ hài lòng với cuộc sống (Phương trình 4-1).

Phương trình 4-1. Một mô hình tuyến tính đơn giản về mức độ hài lòng với cuộc sống

$$\text{life_satisfaction} = \theta_0 + \theta_1 \times \text{GDP_per_capita}$$

Mô hình này chỉ là một hàm tuyến tính của đặc trưng đầu vào `GDP_per_capita`. θ_0 và θ_1 là các tham số của mô hình.

Nói chung, một mô hình tuyến tính đưa ra dự đoán bằng cách đơn giản tính tổng có trọng số của các đặc trưng đầu vào, cộng với một hằng số gọi là *thuật ngữ thiên vị* (còn gọi là *thuật ngữ cắt*), như được thể hiện trong Phương trình 4-2.

Phương trình 4-2. Dự đoán của mô hình hồi quy tuyến tính

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

Trong phương trình này:

- $\hat{y}$ là giá trị dự đoán.
- n là số lượng đặc trưng.
- x_i là giá trị của đặc trưng thứ i.
- θ_j là tham số mô hình thứ j, bao gồm hằng số dịch chuyển θ_0 và các hệ số trọng số của các đặc trưng $\theta_1, \theta_2, \cdots, \theta_n$.

Điều này có thể được viết một cách ngắn gọn hơn nhiều bằng cách sử dụng dạng vectơ, như được thể hiện trong Phương trình 4-3.

Phương trình 4-3. Dự đoán mô hình hồi quy tuyến tính (dạng vectơ)

$$\hat{y} = h_{\theta}(x) = \theta \cdot x$$

Trong phương trình này:

- h_{θ} là hàm giả thuyết, sử dụng các tham số mô hình $\boldsymbol{\theta}$.
- $\boldsymbol{\theta}$ là *vectơ tham số* của mô hình, chứa hằng số dịch chuyển θ_0 và các hệ số đặc trưng *từ* θ_1 đến θ_n.
- $\mathbf{x}$ là *vectơ đặc trưng* của mẫu, chứa các thành phần từ x_0 đến x_n, trong đó x_0 luôn bằng 1.
- $\boldsymbol{\theta} \cdot \mathbf{x}$ là tích vô hướng của các vectơ $\boldsymbol{\theta}$ và $\mathbf{x}$, bằng $\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \ldots + \theta_n x_n$.



Trong học máy, các vectơ thường được biểu diễn dưới dạng *vectơ cột*, tức là các mảng 2D chỉ có một cột. Nếu $\boldsymbol{\theta}$ và $\mathbf{x}$ là các vectơ cột, thì dự đoán là $y = \theta^{\mathsf{T}} x$, trong đó θ^{T} là phép *chuyển vị* của $\boldsymbol{\theta}$ (một vectơ hàng thay vì vectơ cột) và $\theta^{\mathsf{T}} x$ là phép nhân ma trận giữa θ^{T} và $\mathbf{x}$. Tất nhiên, đây vẫn là cùng một dự đoán, chỉ khác là giờ đây nó được biểu diễn dưới dạng một ma trận một ô thay vì một giá trị vô hướng. Trong cuốn sách này, tôi sẽ sử dụng ký hiệu này để tránh phải chuyển đổi giữa tích chấm và phép nhân ma trận.

Được rồi, đó là mô hình hồi quy tuyến tính — nhưng làm thế nào để huấn luyện nó? Hãy nhớ lại rằng huấn luyện một mô hình có nghĩa là thiết lập các tham số của nó sao cho mô hình phù hợp nhất với tập huấn luyện. Để làm điều này, trước tiên chúng ta cần một thước đo đánh giá mức độ phù hợp (hoặc không phù hợp) của mô hình với dữ liệu huấn luyện. Trong Chương 2, chúng ta đã thấy rằng thước đo hiệu suất phổ biến nhất của một mô hình hồi quy là sai số bình phương trung bình (RMSE) (Phương trình 2-1). Do đó, để huấn luyện một mô hình hồi quy tuyến tính, chúng ta cần tìm giá trị của $\boldsymbol{\theta}$ làm cho RMSE đạt giá trị nhỏ nhất. Trong thực tế, việc tối thiểu hóa sai số bình phương trung bình (MSE) đơn giản hơn so với việc tối thiểu hóa

RMSE, và điều này dẫn đến kết quả tương tự (vì giá trị làm tối thiểu một hàm dương cũng làm tối thiểu căn bậc hai của nó).



Các thuật toán học máy thường tối ưu hóa một hàm mất mát khác trong quá trình huấn luyện so với chỉ số hiệu suất được sử dụng để đánh giá mô hình cuối cùng. Điều này thường là do hàm đó dễ tối ưu hóa hơn và/hoặc vì nó chứa các thành phần bổ sung chỉ cần thiết trong quá trình huấn luyện (ví dụ: để thường quy). Một chỉ số hiệu suất tốt là chỉ số gần nhất có thể với mục tiêu kinh doanh cuối cùng. Một hàm mất mát huấn luyện tốt là hàm dễ tối ưu hóa và có mối tương quan chặt chẽ với chỉ số đánh giá. Ví dụ, các bộ phân loại thường được huấn luyện bằng cách sử dụng một hàm chi phí như hàm log loss (như bạn sẽ thấy ở phần sau của chương này) nhưng lại được đánh giá bằng độ chính xác/độ thu hồi. Hàm log loss dễ tối thiểu hóa, và việc làm như vậy thường sẽ cải thiện độ chính xác/độ thu hồi.

MSE của giả thuyết hồi quy tuyến tính $h_{\boldsymbol{\theta}}$trên tập huấn luyện **X** được tính bằng Công thức 4-4.

Phương trình 4-4. Hàm chi phí MSE cho mô hình hồi quy tuyến tính

$$MSE(\mathbf{X}, h_{\boldsymbol{\theta}}) = \frac{1}{m}\sum_{i=1}^{m}\left(\boldsymbol{\theta}^{\intercal}\mathbf{x}^{(i)} - y^{(i)}\right)^2$$

Hầu hết các ký hiệu này đã được trình bày trong Chương 2 (xem "Ký hiệu" ở trang 45). Điểm khác biệt duy nhất là chúng ta viết $h_{\boldsymbol{\theta}}$ thay vì chỉ *viết h* để làm rõ rằng mô hình được tham số hóa bởi vectơ **θ**. Để đơn giản hóa ký hiệu, chúng ta sẽ chỉ viết MSE(**θ**) thay vì MSE(**X**, $h_{\boldsymbol{\theta}}$).

Phương trình chuẩn

Để tìm giá trị của **θ** làm cho sai số bình phương trung bình (MSE) đạt giá trị nhỏ nhất, tồn tại một *giải pháp dạng đóng* — nói cách khác, một phương trình toán học cho kết quả trực tiếp. Đây được gọi là *phương trình chuẩn* (Phương trình 4-5).

Phương trình 4-5. Phương trình chuẩn

$$\widehat{\boldsymbol{\theta}} = \left(\mathbf{X}^{\intercal}\mathbf{X}\right)^{-1}\mathbf{X}^{\intercal}\mathbf{y}$$

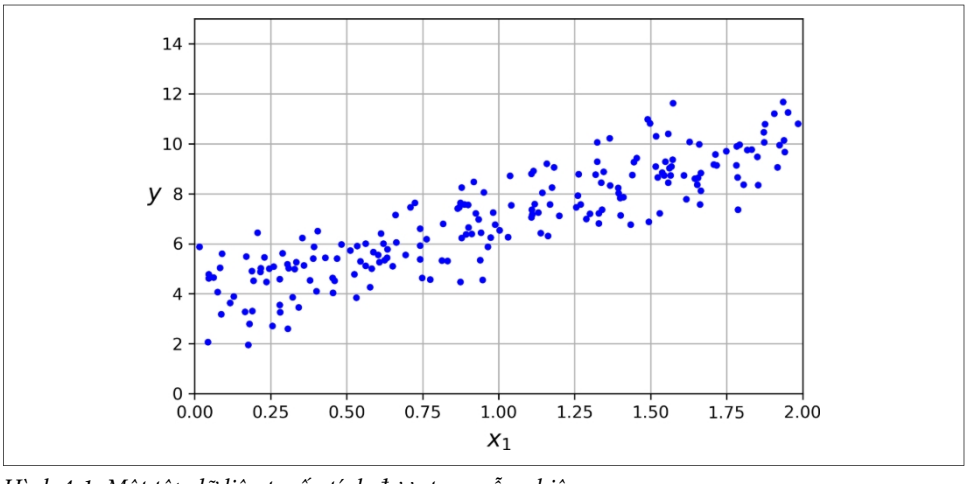
Trong phương trình này:

- $\widehat{\boldsymbol{\theta}}$ là giá trị của **θ** làm tối thiểu hàm chi phí.
- **y** là vectơ các giá trị mục tiêu chứa $y^{(1)}$ đến $y^{(m)}$.

Hãy tạo một số dữ liệu có dạng tuyến tính để kiểm tra phương trình này (Hình 4-1):

```
import numpy as np

rng = np.random.default_rng(seed=42) m
= 200 # số lượng mẫu
X = 2 * rng.random((m, 1)) # vectơ cột
y = 4 + 3 * X + rng.standard_normal((m, 1)) # vectơ cột
```



Hình 4-1. Một tập dữ liệu tuyến tính được tạo ngẫu nhiên

Bây giờ, hãy tính toán $\widehat{\theta}$ bằng phương trình bình thường. Chúng ta sẽ sử dụng hàm `inv()` từ mô-đun đại số tuyến tính của NumPy (`np.linalg`) để tính toán nghịch đảo của ma trận và toán tử `@` để nhân ma trận:

```
from sklearn.preprocessing import add_dummy_feature

X_b = add_dummy_feature(X) # thêm x0 = 1 vào mỗi bản ghi
theta_best = np.linalg.inv(X_b.T @ X_b) @ X_b.T @ y
```



Toán tử `@` thực hiện phép nhân ma trận. Nếu `A` và `B` là các mảng NumPy, thì `A @ B` tương đương với `np.matmul(A, B)`. Nhiều thư viện khác, như TensorFlow, PyTorch và JAX, cũng hỗ trợ toán tử `@`. Tuy nhiên, bạn không thể sử dụng `@` trên các mảng Python thuần túy (tức là danh sách các danh sách).

Hàm mà chúng ta đã sử dụng để tạo dữ liệu là $y = 4 + 3x_1$ + nhiễu Gaussian. Hãy xem phương trình đã tìm ra kết quả gì:

```
>>> theta_best
array([[3.69084138],
       [3.32960458]])
```

Chúng ta đã hy vọng $\theta_0 = 4$ và $\theta_1 = 3$ thay vì $\theta_0 = 3.6908$ và $\theta_1 = 3.3296$. Kết quả khá gần đúng, nhưng nhiễu đã khiến việc khôi phục các tham số chính xác của hàm ban đầu trở nên bất khả thi. Bộ dữ liệu càng nhỏ và càng nhiễu, việc này càng trở nên khó khăn.

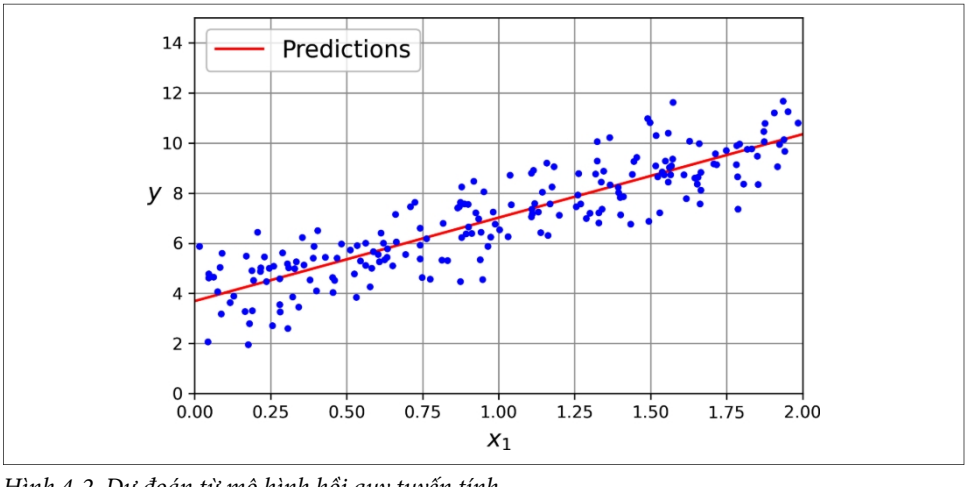
Giờ đây, chúng ta có thể thực hiện dự đoán $\hat{}$ bằng cách sử dụng θ:

```
>>> X_new = np.array([[0], [2]])
>>> X_new_b = add_dummy_feature(X_new) # thêm x0 = 1 vào mỗi bản ghi
>>> y_predict = X_new_b @ theta_best
>>> y_predict array([[
3.69084138],
       [10.35005055]])
```

Hãy vẽ biểu đồ dự đoán của mô hình này (Hình 4-2):

```
import matplotlib.pyplot as plt

plt.plot(X_new, y_predict, "r-", label="Dự đoán")
plt.plot(X, y, "b.")
[...] # làm đẹp hình vẽ: thêm nhãn, trục, lưới và chú thích
plt.show()
```



Hình 4-2. Dự đoán từ mô hình hồi quy tuyến tính

Việc thực hiện hồi quy tuyến tính bằng Scikit-Learn tương đối đơn giản:

```
>>> from sklearn.linear_model import LinearRegression
>>> lin_reg = LinearRegression()
>>> lin_reg.fit(X, y)
>>> lin_reg.intercept_, lin_reg.coef_
(array([3.69084138]), array([[3.32960458]]))
>>> lin_reg.predict(X_new)
array([[ 3.69084138],
       [10.35005055]])
```

Lưu ý rằng Scikit-Learn tách biệt hằng số cắt (`intercept_`) khỏi các hệ số đặc trưng (`coef_`). Lớp `LinearRegression` dựa trên hàm `scipy.linalg.lstsq()` (tên viết tắt của "phương pháp bình phương tối thiểu"), mà bạn có thể gọi trực tiếp:

```
>>> theta_best_svd, residuals, rank, s = np.linalg.lstsq(X_b, y, rcond=1e-6)
>>> theta_best_svd array([[3.69084138],
       [3.32960458]])
```

Hàm này tính toán $\theta = X^{+} \hat{y}$, trong đó X^{+} là *nghịch đảo giả* của **X** (cụ thể là nghịch đảo Moore–Penrose). Bạn có thể sử dụng `np.linalg.pinv()` để tính toán nghịch đảo giả trực tiếp:

```
>>> np.linalg.pinv(X_b) @ y
array([[3.69084138],
       [3.32960458]])
```

Ma trận nghịch đảo giả được tính toán bằng cách sử dụng một kỹ thuật phân tích ma trận tiêu chuẩn gọi là *phân tích giá trị đặc biệt* (SVD), có thể phân tích ma trận tập huấn luyện **X** thành tích của ba ma trận **U, Σ và V**^{T}(xem `numpy.linalg.svd()`). Ma trận nghịch đảo giả được tính theo công thức $X^{+} = V\Sigma^{+}U^{T}$. Để tính ma trận Σ^{+}, thuật toán lấy **ma trận Σ** và đặt tất cả các giá trị nhỏ hơn một ngưỡng rất nhỏ về 0, sau đó thay thế tất cả các giá trị khác 0 bằng nghịch đảo của chúng, và cuối cùng chuyển vị ma trận kết quả. Cách tiếp cận này hiệu quả hơn so với việc tính phương trình bình thường, đồng thời xử lý tốt các trường hợp biên: thực tế, phương trình bình thường có thể không hoạt động nếu ma trận $\mathbf{X}^{T} \times \mathbf{X}$ không khả nghịch (tức là đặc biệt), chẳng hạn như khi $m < n$ hoặc khi một số đặc trưng dư thừa, nhưng ma trận nghịch đảo giả luôn được định nghĩa.

Độ phức tạp tính toán

Phương trình pháp tuyến tính toán ma trận nghịch đảo của $\mathbf{X}^{T} \times \mathbf{X}$, đây là một ma trận $(n + 1) \times (n + 1)$ (trong đó n là số tính năng). *Độ phức tạp tính toán* khi tìm ma trận nghịch đảo của ma trận này thường nằm trong khoảng $O(n^{2.4})$ đến $O(n^{3})$, tùy thuộc vào cách triển khai. Nói cách khác, nếu bạn nhân đôi số lượng đặc trưng, thời gian tính toán sẽ tăng lên khoảng $2^{2.4} = 5.3$ đến $2^{3} = 8$.

Phương pháp SVD được sử dụng bởi lớp `LinearRegression` của Scikit-Learn có độ phức tạp tính toán khoảng $O(n^{2})$. Nếu bạn nhân đôi số lượng đặc trưng, thời gian tính toán sẽ tăng lên khoảng 4 lần.



Cả phương pháp phương trình bình thường lẫn phương pháp SVD đều trở nên rất chậm khi số lượng đặc trưng tăng lên rất lớn (ví dụ: 100.000). Mặt tích cực là cả hai đều có độ phức tạp tuyến tính theo số lượng mẫu trong tập huấn luyện (chúng có *độ phức tạp* $O(m)$), do đó chúng xử lý các tập huấn luyện lớn một cách hiệu quả, miễn là chúng có thể vừa với bộ nhớ.

Ngoài ra, sau khi đã huấn luyện xong mô hình hồi quy tuyến tính (bằng phương trình thông thường hoặc bất kỳ thuật toán nào khác), việc dự đoán diễn ra rất nhanh: độ phức tạp tính toán là tuyến tính đối với cả số lượng mẫu cần dự đoán lẫn số lượng đặc trưng. Nói cách khác, việc dự đoán trên gấp đôi số mẫu (hoặc gấp đôi số đặc trưng) sẽ mất khoảng gấp đôi thời gian.

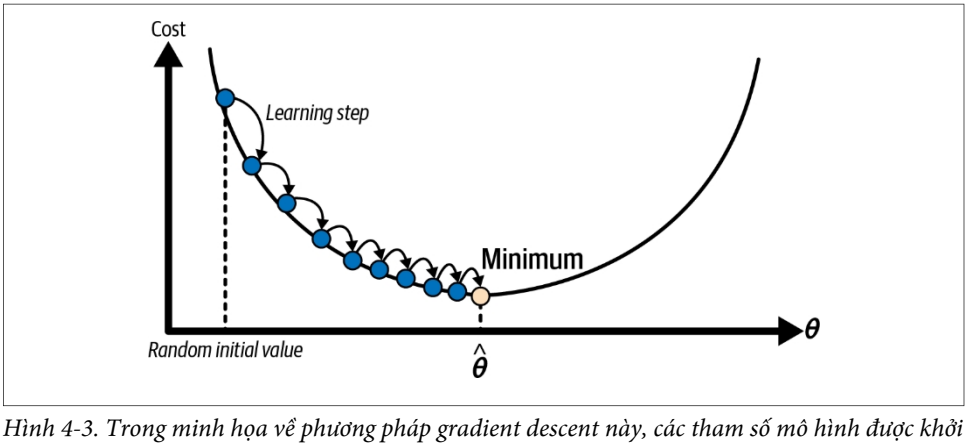
Bây giờ chúng ta sẽ xem xét một cách hoàn toàn khác để huấn luyện mô hình hồi quy tuyến tính, phù hợp hơn với các trường hợp có số lượng đặc trưng lớn hoặc quá nhiều mẫu huấn luyện để chứa vừa trong bộ nhớ.

Hồi quy dốc

Hồi quy theo gradient là một thuật toán tối ưu hóa chung có khả năng tìm ra các giải pháp tối ưu cho một loạt các vấn đề. Ý tưởng chung của hồi quy theo gradient là điều chỉnh các tham số một cách lặp đi lặp lại để giảm thiểu hàm chi phí.

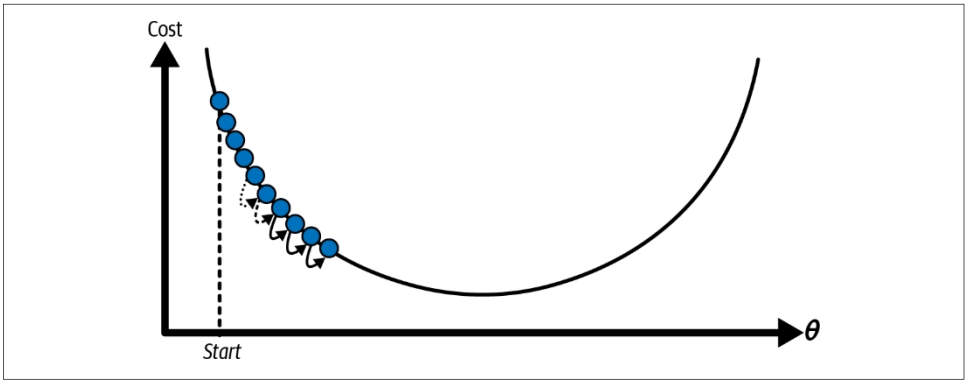
Giả sử bạn bị lạc trong núi giữa làn sương mù dày đặc và chỉ có thể cảm nhận độ dốc của mặt đất dưới chân. Một chiến lược tốt để nhanh chóng xuống đến đáy thung lũng là đi xuống dốc theo hướng có độ dốc lớn nhất. Đây chính xác là cách hoạt động của phương pháp Gradient Descent: nó đo độ dốc cục bộ của hàm sai số liên quan đến vectơ tham số $\boldsymbol{\theta}$, và di chuyển theo hướng độ dốc giảm dần. Khi độ dốc bằng không, bạn đã đạt đến điểm cực tiểu!

Trong thực tế, bạn bắt đầu bằng cách gán các giá trị ngẫu nhiên cho $\boldsymbol{\theta}$ (điều này được gọi là *khởi tạo ngẫu nhiên*). Sau đó, bạn cải thiện nó dần dần, thực hiện từng bước nhỏ một, mỗi bước đều cố gắng giảm hàm chi phí (ví dụ: MSE), cho đến khi thuật toán *hội tụ* đến điểm cực tiểu (xem Hình 4-3).



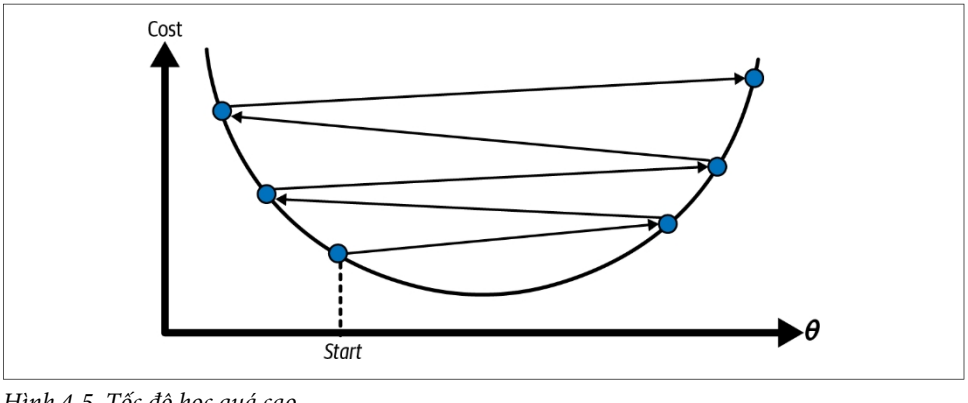
Hình 4-3. Trong minh họa về phương pháp gradient descent này, các tham số mô hình được khởi tạo ngẫu nhiên và được điều chỉnh lặp đi lặp lại để tối thiểu hóa hàm chi phí; kích thước bước học tỷ lệ thuận với độ dốc của hàm chi phí, do đó các bước sẽ dần nhỏ lại khi hàm chi phí tiến gần đến điểm cực tiểu

Một tham số quan trọng trong phương pháp gradient descent là kích thước của các bước, được xác định bởi siêu tham số *tốc độ học*. Nếu tốc độ học quá nhỏ, thuật toán sẽ phải trải qua nhiều lần lặp lại để hội tụ, điều này sẽ mất nhiều thời gian (xem Hình 4-4).



Hình 4-4. Tốc độ học quá nhỏ

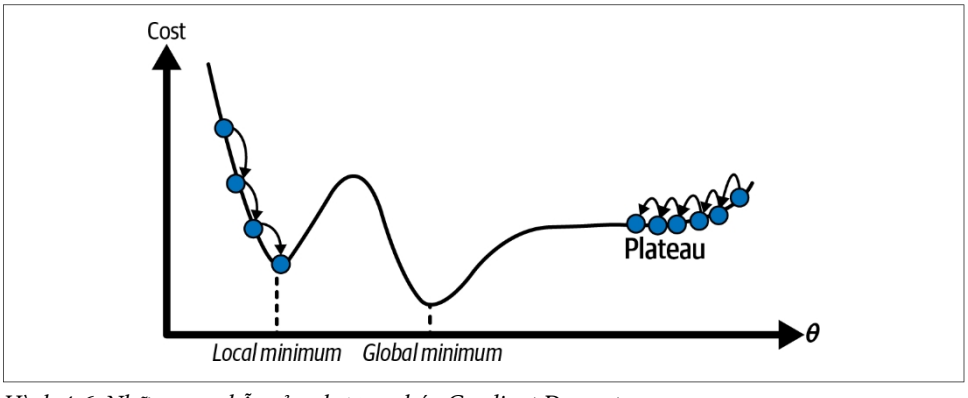
Mặt khác, nếu tốc độ học quá cao, bạn có thể "nhảy" qua thung lũng và kết thúc ở phía bên kia, thậm chí có thể ở vị trí cao hơn so với trước đó. Điều này có thể khiến thuật toán phân kỳ, với các giá trị ngày càng lớn, dẫn đến việc không tìm được giải pháp tối ưu (xem Hình 4-5).



Hình 4-5. Tốc độ học quá cao

Ngoài ra, không phải tất cả các hàm chi phí đều trông giống như những cái bát đẹp và đều đặn. Có thể có lỗ, gờ, cao nguyên và tất cả các loại địa hình không đều, khiến việc hội tụ đến giá trị tối thiểu trở nên khó khăn. Hình 4-6 cho thấy hai thách thức chính của phương pháp giảm dốc. Nếu khởi tạo ngẫu nhiên bắt đầu thuật toán ở bên trái, thì nó sẽ hội tụ đến giá *trị tối thiểu cục bộ*, không tốt bằng giá *trị tối thiểu toàn cục*. Nếu bắt đầu ở bên phải,

thì sẽ mất rất nhiều thời gian để vượt qua vùng cao nguyên. Và nếu bạn dừng quá sớm, bạn sẽ không bao giờ đạt được điểm cực tiểu toàn cục.



Hình 4-6. Những cạm bẫy của phương pháp Gradient Descent

May mắn thay, hàm chi phí MSE cho mô hình hồi quy tuyến tính lại là một *hàm lồi*, có nghĩa là nếu bạn chọn bất kỳ hai điểm nào trên đường cong, đoạn thẳng nối chúng sẽ không bao giờ nằm dưới đường cong. Điều này ngụ ý rằng không có điểm cực tiểu cục bộ, mà chỉ có một điểm cực tiểu toàn cục. Đây cũng là một hàm liên tục với độ dốc không bao giờ thay đổi đột ngột.[2] Hai sự thật này mang lại một hệ quả quan trọng: phương pháp gradient descent được đảm bảo sẽ tiến gần đến điểm cực tiểu toàn cục một cách tùy ý (nếu bạn chờ đủ lâu và nếu tốc độ học không quá cao).

Mặc dù hàm chi phí có hình dạng như một cái bát, nó có thể trở thành một cái bát kéo dài nếu các đặc trưng có độ lớn rất khác nhau. Hình 4-7 minh họa thuật toán gradient descent trên tập huấn luyện nơi các đặc trưng 1 và 2 có cùng độ lớn (bên trái), và trên tập huấn luyện nơi đặc trưng 1 có giá trị nhỏ hơn nhiều so với đặc trưng 2 (bên phải).[3]

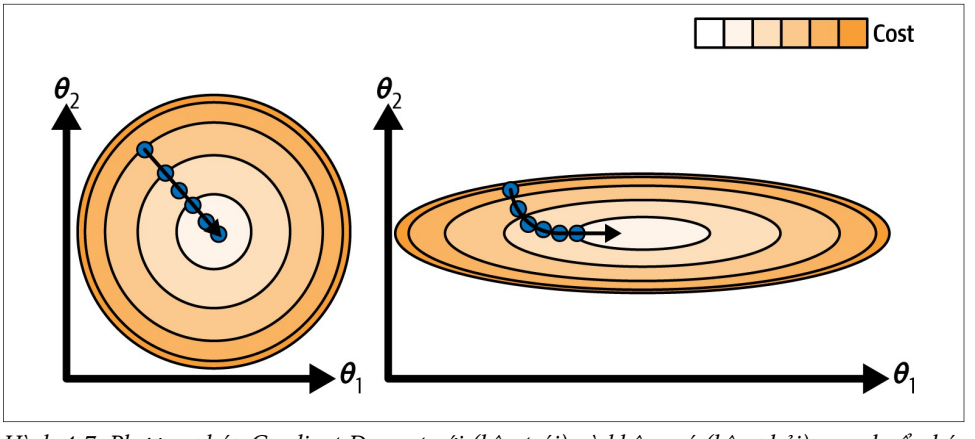
Như bạn có thể thấy, ở bên trái, thuật toán gradient descent đi thẳng về phía điểm cực tiểu, do đó đạt được nó nhanh chóng, trong khi ở bên phải, nó ban đầu di chuyển theo hướng gần như vuông góc với hướng của điểm cực tiểu toàn cục, và kết thúc bằng một hành trình dài xuống một thung lũng gần như bằng phẳng. Cuối cùng nó sẽ đạt được điểm cực tiểu, nhưng sẽ mất rất nhiều thời gian.

Sơ đồ này cũng minh họa thực tế rằng việc huấn luyện mô hình có nghĩa là tìm kiếm một tổ hợp các tham số mô hình sao cho hàm chi phí đạt giá trị nhỏ nhất (trên tập huấn luyện). Đây là một quá trình tìm kiếm trong *không gian tham số* của mô hình. Mô hình có càng nhiều tham số, không gian này càng có nhiều chiều, và quá trình tìm kiếm càng khó khăn: tìm kiếm

2 Về mặt kỹ thuật, đạo hàm của nó là *liên tục theo Lipschitz*.

3 Vì đặc trưng 1 có giá trị nhỏ hơn, nên cần một sự thay đổi lớn hơn trong θ_1 để ảnh hưởng đến hàm chi phí, đó là lý do tại sao cái bát bị kéo dài dọc theo trục θ_1.

việc tìm một cây kim trong đống cỏ khô 300 chiều khó khăn hơn nhiều so với trong không gian 3 chiều. May mắn thay, do hàm chi phí là lồi trong trường hợp hồi quy tuyến tính, nên cây kim chỉ đơn giản nằm ở đáy cái bát.



Hình 4-7. Phương pháp Gradient Descent với (bên trái) và không có (bên phải) quy chuẩn hóa đặc trưng



Khi sử dụng phương pháp Gradient Descent, bạn nên đảm bảo rằng tất cả các đặc trưng đều có cùng một tỷ lệ (ví dụ: sử dụng lớp `StandardScaler` của Scikit-Learn), nếu không thì sẽ mất nhiều thời gian hơn để hội tụ.

Hồi quy theo gradient theo lô

Hầu hết các mô hình đều có nhiều hơn một tham số mô hình. Do đó, để triển khai phương pháp gradient descent, bạn cần tính toán gradient của hàm chi phí đối với từng tham số mô hình θ_j. Nói cách khác, bạn cần tính toán hàm chi phí sẽ thay đổi bao nhiêu nếu bạn thay đổi $\theta_j$chỉ một chút. Đây được gọi là *đạo hàm riêng*. Nó giống như việc hỏi: "Độ dốc của ngọn núi hướng về phía đông là bao nhiêu?" rồi hỏi lại câu hỏi tương tự khi nhìn về phía bắc (và cứ thế đối với tất cả các chiều khác, nếu bạn có thể hình dung một vũ trụ có hơn ba chiều). Phương trình 4-6 tính đạo hàm riêng của MSE theo tham số θ_j, được ký hiệu là $\partial \text{MSE}(\mathbf{\theta}) / \partial\theta_j$.

Phương trình 4-6. Đạo hàm riêng của hàm chi phí

$$\frac{\partial}{\partial\theta_j}\text{MSE}(\theta) = \frac{2}{m}\sum_{i=1}^{m}\left(\theta^{T}\mathbf{x}^{(i)} - y^{(i)}\right)x_j^{(i)}$$

Thay vì tính từng đạo hàm riêng lẻ, bạn có thể sử dụng Phương trình 4-7 để tính tất cả cùng một lúc. Vectơ gradient, được ký hiệu là $\nabla_{\theta}\,\mathrm{MSE}(\boldsymbol{\theta})$, chứa tất cả các đạo hàm riêng của hàm chi phí (mỗi tham số mô hình tương ứng với một đạo hàm).

Phương trình 4-7. Vectơ gradient của hàm chi phí

$$\nabla_{\theta}\,\mathrm{MSE}(\theta) = \begin{pmatrix} \frac{\partial}{\partial\theta_0}\mathrm{MSE}(\theta) \\ \frac{\partial}{\partial\theta_1}\mathrm{MSE}(\theta) \\ \vdots \\ \frac{\partial}{\partial\theta_n}\mathrm{MSE}(\theta) \end{pmatrix} = \frac{2}{m}X^{\intercal}(X\theta - y)$$



Lưu ý rằng công thức này liên quan đến các phép tính trên toàn bộ tập huấn luyện **X**, tại mỗi bước giảm dốc độ dốc! Đây là lý do tại sao thuật toán này được gọi là *giảm dốc độ dốc theo lô*: nó sử dụng toàn bộ lô dữ liệu huấn luyện tại mỗi bước (thực ra, *giảm dốc độ dốc toàn phần* có lẽ sẽ là một tên gọi phù hợp hơn). Kết quả là, nó cực kỳ chậm trên các tập huấn luyện rất lớn (chúng ta sẽ xem xét một số thuật toán giảm dốc độ dốc nhanh hơn nhiều trong thời gian ngắn). Tuy nhiên, phương pháp gradient descent mở rộng tốt theo số lượng đặc trưng; việc huấn luyện mô hình hồi quy tuyến tính khi có hàng trăm nghìn đặc trưng sẽ nhanh hơn nhiều khi sử dụng gradient descent so với phương trình bình thường hoặc phân tích SVD.

Khi đã có vectơ gradient, vốn chỉ hướng lên dốc, chỉ cần đi theo hướng ngược lại để đi xuống dốc. Điều này có nghĩa là trừ $\nabla_{\theta}\,\mathrm{MSE}(\boldsymbol{\theta})$ khỏi $\boldsymbol{\theta}$. Đây là lúc tốc độ học η phát huy tác dụng:[4] nhân vectơ gradient với η để xác định độ lớn của bước đi xuống dốc (Phương trình 4-8).

Phương trình 4-8. Bước gradient descent

$$\theta^{(\text{bước tiếp theo})} = \theta - \eta\nabla_{\theta}\mathrm{MSE}(\theta)$$

Hãy xem xét một cách triển khai nhanh thuật toán này:

```
eta = 0,1 # tốc độ học
n_epochs = 1000
m = len(X_b) # số mẫu

rng = np.random.default_rng(seed=42)
theta = rng.standard_normal((2, 1)) # các tham số mô hình được khởi tạo ngẫu nhiên
```

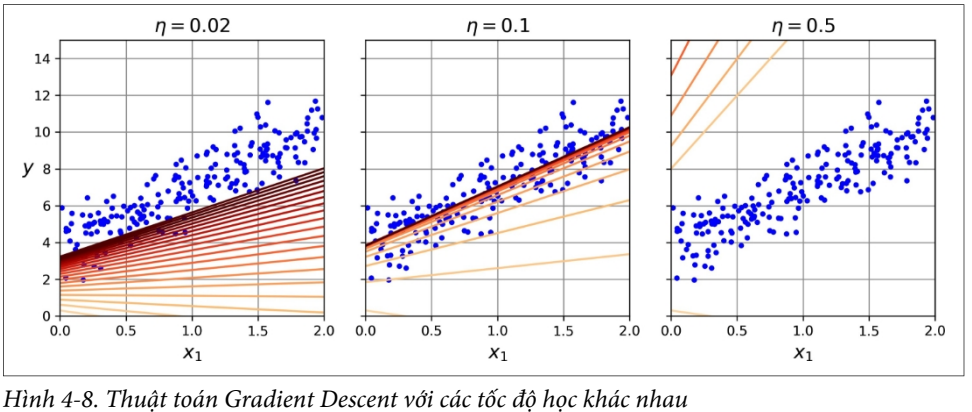
4 Eta (η) là chữ cái thứ bảy trong bảng chữ cái Hy Lạp.

```
for epoch in range(n_epochs):
    gradients = 2 / m * X_b.T @ (X_b @ theta - y) theta
    = theta - eta * gradients
```

Cũng không quá khó nhỉ! Mỗi lần lặp lại trên tập huấn luyện được gọi là một *epoch*. Hãy xem kết quả của `theta`:

```
>>> theta
array([[3.69084138],
       [3.32960458]])
```

Này, đó chính xác là kết quả mà phương trình bình thường tìm được! Phương pháp gradient descent đã hoạt động hoàn hảo. Nhưng nếu bạn sử dụng một tốc độ học (`eta`) khác thì sao? Hình 4-8 cho thấy 20 bước đầu tiên của phương pháp gradient descent khi sử dụng ba tốc độ học khác nhau. Đường ở dưới cùng của mỗi đồ thị biểu thị điểm khởi đầu ngẫu nhiên, sau đó mỗi epoch được biểu thị bằng một đường ngày càng đậm hơn.



Hình 4-8. Thuật toán Gradient Descent với các tốc độ học khác nhau

Ở bên trái, tốc độ học quá thấp: thuật toán cuối cùng sẽ đạt được giải pháp, nhưng sẽ mất rất nhiều thời gian. Ở giữa, tốc độ học trông khá tốt: chỉ sau vài epoch, nó đã hội tụ đến giải pháp. Ở bên phải, tốc độ học quá cao: thuật toán phân kỳ, nhảy lung tung khắp nơi và thực tế là càng ngày càng xa rời giải pháp ở mỗi bước.

Để tìm ra tốc độ học tập phù hợp, bạn có thể sử dụng phương pháp tìm kiếm lưới (xem Chương 2). Tuy nhiên, bạn có thể muốn giới hạn số epoch để phương pháp tìm kiếm lưới có thể loại bỏ các mô hình mất quá nhiều thời gian để hội tụ.

Bạn có thể thắc mắc làm thế nào để thiết lập số epoch. Nếu số epoch quá thấp, bạn vẫn sẽ còn cách xa giải pháp tối ưu khi thuật toán dừng lại; nhưng nếu số epoch quá cao, bạn sẽ lãng phí thời gian trong khi các tham số mô hình không còn thay đổi nữa. Một giải pháp đơn giản là thiết lập một số epoch rất lớn nhưng ngắt thuật toán khi vectơ gradient trở nên rất nhỏ — tức là khi độ lớn của nó nhỏ hơn một

số rất nhỏ ε (gọi là độ *dung sai*)—bởi vì điều này xảy ra khi phương pháp gradient descent đã (gần như) đạt đến điểm cực tiểu.

Tốc độ hội tụ

Khi hàm chi phí là lồi và độ dốc của nó không thay đổi đột ngột (như trường hợp của hàm chi phí MSE), phương pháp gradient lùi theo lô với tốc độ học cố định cuối cùng sẽ hội tụ đến giải pháp tối ưu, nhưng bạn có thể phải chờ một thời gian: có thể mất $O(1/\varepsilon)$ lần lặp để đạt được điểm cực tiểu trong khoảng ε, tùy thuộc vào hình dạng của hàm chi phí. Nếu bạn chia độ dung sai cho 10 để có được giải pháp chính xác hơn, thì thuật toán có thể phải chạy lâu hơn khoảng 10 lần.

Hồi quy Gradient Ngẫu nhiên

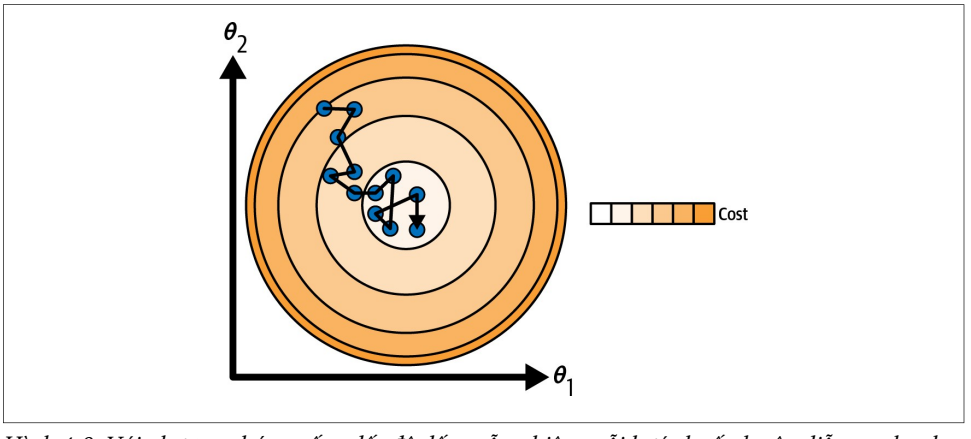
Vấn đề chính của phương pháp gradient descent theo lô là việc nó sử dụng toàn bộ tập huấn luyện để tính toán gradient ở mỗi bước, điều này khiến thuật toán trở nên rất chậm khi tập huấn luyện có quy mô lớn. Ngược lại, *phương pháp gradient descent ngẫu nhiên* chọn một mẫu ngẫu nhiên trong tập huấn luyện ở mỗi bước và tính toán gradient dựa trên duy nhất mẫu đó. Rõ ràng, việc xử lý từng mẫu một giúp thuật toán nhanh hơn nhiều vì lượng dữ liệu cần xử lý ở mỗi lần lặp rất ít. Điều này cũng cho phép huấn luyện trên các tập huấn luyện khổng lồ, vì chỉ cần một mẫu duy nhất được lưu trong bộ nhớ ở mỗi lần lặp (hạ độ dốc ngẫu nhiên có thể được triển khai dưới dạng thuật toán ngoài bộ nhớ; xem Chương 1).

Mặt khác, do bản chất ngẫu nhiên (tức là ngẫu nhiên) của nó, thuật toán này ít đều đặn hơn nhiều so với phương pháp gradient descent theo lô: thay vì giảm dần một cách nhẹ nhàng cho đến khi đạt đến cực tiểu, hàm chi phí sẽ dao động lên xuống, chỉ giảm trung bình. Theo thời gian, nó sẽ tiến rất gần đến điểm cực tiểu, nhưng một khi đến đó, nó sẽ tiếp tục dao động xung quanh, không bao giờ ổn định (xem Hình 4-9). Khi thuật toán dừng lại, các giá trị tham số cuối cùng sẽ tốt, nhưng không phải là tối ưu.

Khi hàm chi phí rất không đều (như trong Hình 4-6), điều này thực sự có thể giúp thuật toán thoát khỏi các điểm cực tiểu cục bộ, do đó thuật toán gradient ngẫu nhiên có cơ hội tìm được điểm cực tiểu toàn cục cao hơn so với thuật toán gradient theo lô.

Do đó, tính ngẫu nhiên có lợi trong việc thoát khỏi các điểm cực tiểu cục bộ, nhưng lại bất lợi vì nó khiến thuật toán không bao giờ ổn định tại điểm cực tiểu toàn cục. Một giải pháp cho tình huống này là giảm dần tốc độ học. Các bước ban đầu có giá trị lớn (giúp tiến triển nhanh và thoát khỏi các điểm cực tiểu cục bộ), sau đó ngày càng nhỏ dần, cho phép thuật toán ổn định tại điểm cực tiểu toàn cục. Quá trình này tương tự như thuật toán *làm nguội mô phỏng* (*simulated annealing*), một thuật toán lấy cảm hứng từ quá trình làm nguội trong luyện kim, nơi kim loại nóng chảy được làm nguội từ từ. Hàm xác định tốc độ học ở mỗi lần lặp được gọi là *lịch trình học*. Nếu tốc độ học được giảm quá

nhanh, bạn có thể bị kẹt tại điểm cực tiểu cục bộ, hoặc thậm chí bị "đóng băng" ở giữa chừng trên đường đến điểm cực tiểu. Nếu tốc độ học được giảm quá chậm, bạn có thể dao động quanh điểm cực tiểu trong thời gian dài và kết thúc với một giải pháp không tối ưu nếu dừng huấn luyện quá sớm.



Hình 4-9. Với phương pháp xuống dốc độ dốc ngẫu nhiên, mỗi bước huấn luyện diễn ra nhanh hơn nhiều nhưng cũng mang tính ngẫu nhiên cao hơn nhiều so với khi sử dụng phương pháp xuống dốc độ dốc theo lô

Mã này triển khai phương pháp gradient ngẫu nhiên bằng cách sử dụng một lịch trình học đơn giản:

```
n_epochs = 50
t0, t1 = 5, 50 # siêu tham số lịch trình học

            def lịch_học(t):
      return t0 / (t + t1)

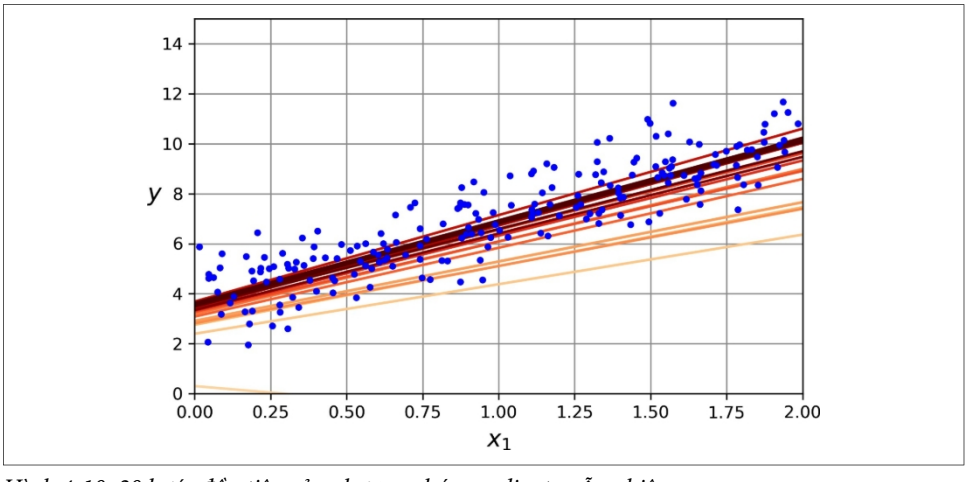
rng = np.random.default_rng(seed=42)
theta = rng.standard_normal((2, 1)) # các tham số mô hình được khởi tạo ngẫu nhiên

for epoch in range(n_epochs):
    for iteration in range(m): random_index
        = rng.integers(m)
        xi = X_b[random_index : random_index + 1] yi
        = y[random_index : random_index + 1]
        gradients = 2 * xi.T @ (xi @ theta - yi) # đối với SGD, không chia cho m
        eta = learning_schedule(epoch * m + iteration) theta
        = theta - eta * gradients
```

Theo quy ước, chúng ta lặp lại theo các vòng gồm *m* lần lặp; mỗi vòng được gọi là một *epoch*, như đã đề cập trước đó. Trong khi mã gradient descent theo lô lặp lại 1.000 lần qua toàn bộ tập huấn luyện, mã này chỉ lặp lại qua tập huấn luyện 50 lần và đạt được một giải pháp khá tốt:

```
>>> theta
array([[3.69826475],
       [3.30748311]])
```

Hình 4-10 cho thấy 20 bước đầu tiên của quá trình huấn luyện (lưu ý rằng các bước này không đều đặn như thế nào).



Hình 4-10. 20 bước đầu tiên của phương pháp gradient ngẫu nhiên

Lưu ý rằng vì các mẫu được chọn ngẫu nhiên, một số mẫu có thể được chọn nhiều lần trong mỗi epoch, trong khi những mẫu khác có thể không được chọn lần nào. Nếu bạn muốn đảm bảo thuật toán xử lý mọi mẫu trong mỗi epoch, một phương pháp khác là trộn ngẫu nhiên tập huấn luyện (đảm bảo trộn cả các đặc trưng đầu vào và nhãn cùng lúc), sau đó xử lý từng mẫu một, rồi trộn lại, và cứ thế tiếp tục. Tuy nhiên, phương pháp này phức tạp hơn và thường không cải thiện kết quả.



Khi sử dụng phương pháp gradient ngẫu nhiên (SGD), các mẫu huấn luyện phải độc lập và có cùng phân phối (IID) để đảm bảo rằng các tham số sẽ được kéo về phía cực tiểu toàn cục, trung bình. Một cách đơn giản để đảm bảo điều này là xáo trộn các mẫu trong quá trình huấn luyện (ví dụ: chọn từng mẫu ngẫu nhiên, hoặc xáo trộn tập huấn luyện vào đầu mỗi epoch). Nếu bạn không xáo trộn các mẫu — ví dụ: nếu các mẫu được sắp xếp theo nhãn — thì SGD sẽ bắt đầu bằng việc tối ưu hóa cho một nhãn, sau đó là nhãn tiếp theo, và cứ thế tiếp tục, và nó sẽ không ổn định gần điểm cực tiểu toàn cục.

Để thực hiện hồi quy tuyến tính bằng phương pháp GD ngẫu nhiên với Scikit-Learn, bạn có thể sử dụng lớp `SGDRegressor`, lớp này mặc định tối ưu hóa hàm chi phí MSE. Mã sau đây chạy tối đa 1.000 epoch (`max_iter`) hoặc cho đến khi độ mất giảm ít hơn 10^{-5} (`tol`) trong vòng 100 epoch (`n_iter_no_change`). Nó bắt đầu với tốc độ học là 0,01 (`eta0`), sử dụng lịch trình học mặc định (khác với

chúng ta đã sử dụng). Cuối cùng, chương trình không sử dụng bất kỳ phương pháp chính quy hóa nào (`penalty=None`; sẽ có thêm chi tiết về vấn đề này ngay sau đây):

```
from sklearn.linear_model import SGDRegressor

sgd_reg = SGDRegressor(max_iter=1000, tol=1e-5, penalty=None, eta0=0.01,
                       n_iter_no_change=100, random_state=42) sgd_reg.fit(X,
y.ravel()) # Sử dụng y.ravel() vì hàm fit() yêu cầu dữ liệu mục tiêu 1 chiều
```

Một lần nữa, bạn sẽ tìm thấy một giải pháp khá gần với giải pháp được trả về bởi phương trình chuẩn:

```
>>> sgd_reg.intercept_, sgd_reg.coef_
(array([3.68899733]), array([3.33054574]))
```



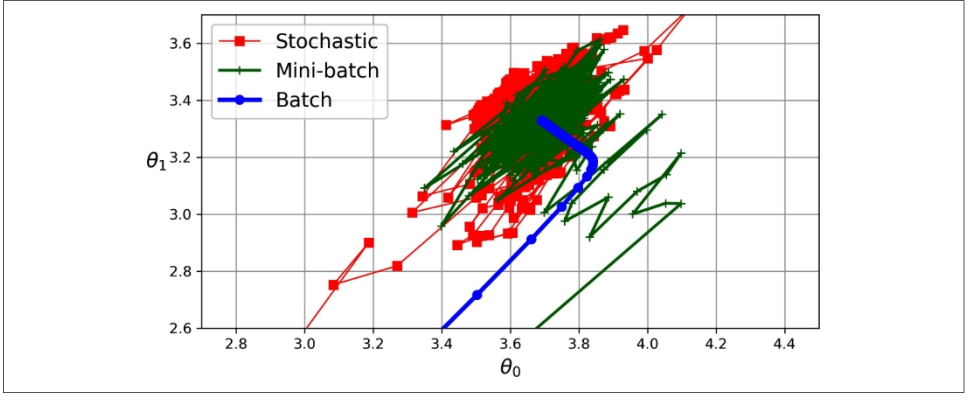
Tất cả các mô hình ước lượng trong Scikit-Learn đều có thể được huấn luyện bằng phương thức `fit()`, nhưng một số mô hình còn có phương thức `partial_fit()` mà bạn có thể gọi để thực hiện một vòng huấn luyện trên một hoặc nhiều bản ghi (phương thức này bỏ qua các siêu tham số như `max_iter` hoặc `tol`). Việc gọi lặp lại `partial_fit()` sẽ dần dần huấn luyện mô hình. Điều này hữu ích khi bạn cần kiểm soát chặt chẽ hơn quá trình huấn luyện. Các mô hình khác thay vào đó có tham số siêu (hyperparameter) `warm_start` (và một số mô hình có cả hai): nếu bạn đặt `warm_start=True`, việc gọi phương thức `fit()` trên một mô hình đã được huấn luyện sẽ không đặt lại mô hình; nó sẽ chỉ tiếp tục huấn luyện từ điểm dừng lại, tuân thủ các tham số siêu như `max_iter` và `tol`. Lưu ý rằng `fit()` đặt lại bộ đếm lặp được sử dụng bởi lịch trình học, trong khi `partial_fit()` thì không.

Hồi quy Gradient theo lô nhỏ

Thuật toán gradient descent cuối cùng mà chúng ta sẽ xem xét được gọi là *gradient descent theo lô nhỏ (mini-batch gradient descent)*. Nó khá đơn giản một khi bạn đã hiểu về gradient descent theo lô (batch) và gradient descent ngẫu nhiên (stochastic): tại mỗi bước, thay vì tính toán gradient dựa trên toàn bộ tập huấn luyện (như trong gradient descent theo lô) hoặc dựa trên chỉ một mẫu (như trong gradient descent ngẫu nhiên), gradient descent theo lô nhỏ tính toán gradient trên các tập con ngẫu nhiên nhỏ của các mẫu được gọi là *lô nhỏ (mini-batches)*. Ưu điểm chính của thuật toán Gradient Descent theo lô nhỏ so với thuật toán Gradient Descent ngẫu nhiên là bạn có thể đạt được sự cải thiện hiệu suất nhờ khả năng tăng tốc phần cứng cho các phép toán ma trận, đặc biệt khi sử dụng *bộ xử lý đồ họa* (GPU).

Quá trình di chuyển của thuật toán trong không gian tham số ít dao động hơn so với GD ngẫu nhiên, đặc biệt khi sử dụng các mini-batch có kích thước khá lớn. Do đó, GD mini-batch sẽ di chuyển gần điểm cực tiểu hơn so với GD ngẫu nhiên — nhưng nó có thể gặp khó khăn hơn trong việc thoát khỏi các điểm cực tiểu cục bộ (trong trường hợp các bài toán bị ảnh hưởng bởi các điểm cực tiểu cục bộ, khác với hồi quy tuyến tính sử dụng hàm chi phí MSE). Hình 4-11 minh họa các đường đi của ba thuật toán gradient descent trong không gian tham số trong quá trình huấn luyện. Tất cả đều kết thúc gần điểm cực tiểu, nhưng đường đi của thuật toán gradient descent theo lô (batch GD) thực sự

dừng lại tại điểm cực tiểu, trong khi cả GD ngẫu nhiên và GD mini-batch vẫn tiếp tục di chuyển xung quanh. Tuy nhiên, đừng quên rằng GD theo lô mất rất nhiều thời gian cho mỗi bước, và GD ngẫu nhiên cùng GD mini-batch cũng sẽ đạt đến điểm cực tiểu nếu bạn sử dụng lịch trình học tập phù hợp.



Hình 4-11. Các đường đi của thuật toán Gradient Descent trong không gian tham số

Bảng 4-1 so sánh các thuật toán mà chúng ta đã thảo luận cho đến nay đối với hồi quy tuyến tính[5] (nhắc lại rằng *m* là số mẫu huấn luyện và *n* là số đặc trưng).

Bảng 4-1. So sánh các thuật toán cho hồi quy tuyến tính

Thuật toán	*m* lớn	Hỗ trợ ngoài bộ nhớ	*n* lớn	Siêu tham số	Cần điều chỉnh tỷ lệ	Scikit-Learn
Phương trình thường	Nhanh	Không	Chậm	0	Không	Không áp dụng
SVD	Nhanh	Không	Chậm	0	Không	`LinearRegression`
GD theo lô	Chậm	Không	Nhanh	2	Có	Không áp dụng
GD ngẫu nhiên	Nhanh	Có	Nhanh	≥2	Có	`SGDRegressor`
GD theo lô nhỏ	Nhanh	Có	Nhanh	≥2	Có	Không áp dụng

Sau khi huấn luyện, hầu như không có sự khác biệt: tất cả các thuật toán này đều tạo ra các mô hình rất giống nhau và đưa ra dự đoán theo cách hoàn toàn giống nhau.

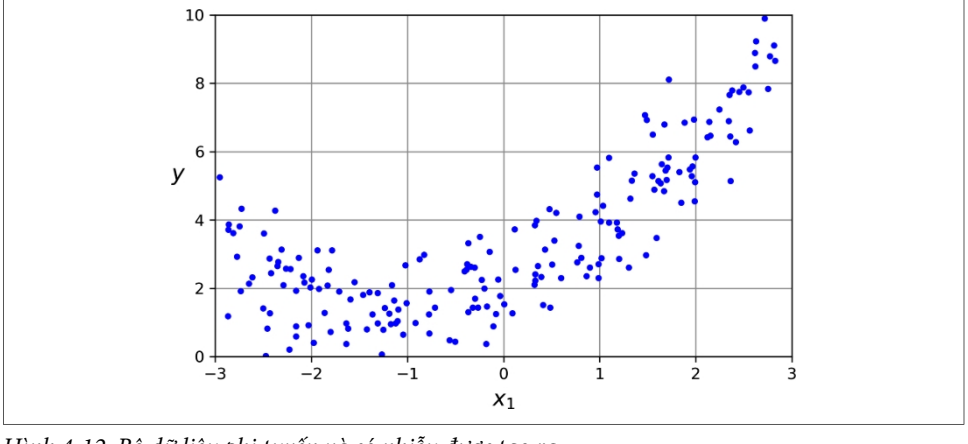
5 Trong khi phương trình bình thường chỉ có thể thực hiện hồi quy tuyến tính, các thuật toán gradient descent có thể được sử dụng để huấn luyện nhiều mô hình khác, như bạn sẽ thấy.

Hồi quy đa thức

Điều gì sẽ xảy ra nếu dữ liệu của bạn phức tạp hơn một đường thẳng? Đáng ngạc nhiên là bạn có thể sử dụng mô hình tuyến tính để phù hợp với dữ liệu phi tuyến tính. Một cách đơn giản để làm điều này là thêm các lũy thừa của từng đặc trưng làm các đặc trưng mới, sau đó huấn luyện mô hình tuyến tính trên tập hợp các đặc trưng mở rộng này. Kỹ thuật này được gọi là *hồi quy đa thức*.

Hãy xem một ví dụ. Đầu tiên, chúng ta sẽ tạo ra một số dữ liệu phi tuyến (xem Hình 4-12), dựa trên một *phương trình bậc hai* đơn giản — đó là phương trình có dạng $y = ax^2 + bx + c$ — cộng với một ít nhiễu:

```
rng = np.random.default_rng(seed=42) m
= 200 # số lượng bản ghi
X = 6 * rng.random((m, 1)) - 3
y = 0.5 * X ** 2 + X + 2 + rng.standard_normal((m, 1))
```



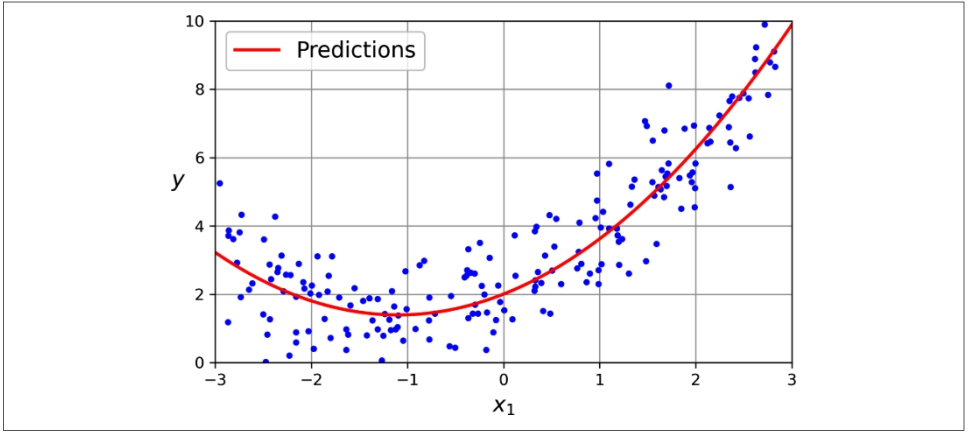
Hình 4-12. Bộ dữ liệu phi tuyến và có nhiễu được tạo ra

Rõ ràng, một đường thẳng sẽ không bao giờ phù hợp với dữ liệu này. Vì vậy, hãy sử dụng lớp `PolynomialFeatures` của Scikit-Learn để biến đổi dữ liệu huấn luyện của chúng ta, thêm bình phương (đa thức bậc hai) của mỗi đặc trưng trong tập huấn luyện làm một đặc trưng mới (trong trường hợp này chỉ có một đặc trưng):

```
>>> from sklearn.preprocessing import PolynomialFeatures
>>> poly_features = PolynomialFeatures(degree=2, include_bias=False)
>>> X_poly = poly_features.fit_transform(X)
>>> X[0] mảng([1.64373629])
>>> X_poly[0] array([1.64373629,
2.701869 ])
```

`X_poly` hiện chứa đặc trưng ban đầu của `X` cộng với bình phương của đặc trưng này. Bây giờ chúng ta có thể vừa khít mô hình `LinearRegression` với dữ liệu huấn luyện mở rộng này (Hình 4-13):

```
>>> lin_reg = LinearRegression()
>>> lin_reg.fit(X_poly, y)
>>> lin_reg.intercept_, lin_reg.coef_ (mảng([2.00540719]),
mảng([[1.11022126, 0.50526985]]))
```



Hình 4-13. Dự đoán của mô hình hồi quy đa thức

Không tệ: mô hình ước tính $y = 0.^{56}x^{2} + 0.93x + 1.78$ trong khi thực tế, phương trình ban đầu là $y = 0{,}5x^{2} + 1{,}0x + 2{,}0$ cộng với nhiễu Gauss.

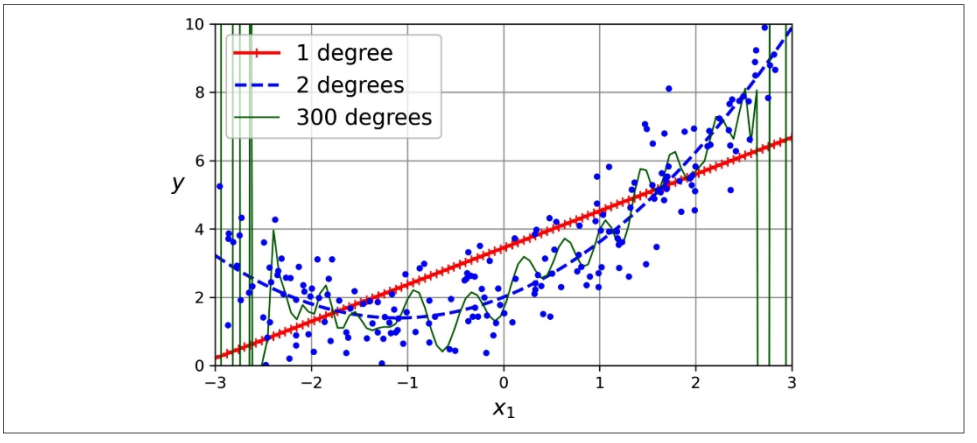
Lưu ý rằng khi có nhiều đặc trưng, hồi quy đa thức có khả năng tìm ra mối quan hệ giữa các đặc trưng, điều mà mô hình hồi quy tuyến tính đơn thuần không thể làm được. Điều này trở nên khả thi nhờ việc `PolynomialFeatures` cũng cộng tất cả các tổ hợp của các đặc trưng lên đến bậc đã cho. Ví dụ, nếu có hai đặc trưng a và b, `PolynomialFeatures` với `độ = 3` sẽ không chỉ cộng các đặc trưng a^2, a^3, b^2 và b^3, mà còn cộng các tổ hợp ab, a^2b và ab^2.



`Hàm PolynomialFeatures(`*degree=d*`)` chuyển đổi một mảng chứa n đặc trưng thành một mảng chứa $(n + d)! / d!n!$ đặc trưng, trong đó $n!$ là *giai thừa* của n, bằng $1 \times 2 \times 3 \times \cdots \times n$. Hãy cẩn thận với hiện tượng "nổ tung" về số lượng đặc trưng!

Đường cong học tập

Nếu bạn thực hiện hồi quy đa thức bậc cao, bạn có thể sẽ phù hợp với dữ liệu huấn luyện tốt hơn nhiều so với hồi quy tuyến tính đơn thuần. Ví dụ, Hình 4-14 áp dụng mô hình đa thức bậc 300 cho dữ liệu huấn luyện trước đó và so sánh kết quả với mô hình tuyến tính thuần túy và mô hình bậc hai (đa thức bậc hai). Hãy chú ý cách mô hình đa thức bậc 300 dao động để tiếp cận các mẫu huấn luyện càng gần càng tốt.



Hình 4-14. Hồi quy đa thức bậc cao

Mô hình hồi quy đa thức bậc cao này đang bị quá khớp dữ liệu huấn luyện một cách nghiêm trọng, trong khi mô hình tuyến tính lại bị thiếu khớp. Mô hình có khả năng tổng quát hóa tốt nhất trong trường hợp này là mô hình bậc hai, điều này hợp lý vì dữ liệu được tạo ra bằng mô hình bậc hai. Tuy nhiên, nói chung bạn sẽ không biết hàm nào đã tạo ra dữ liệu, vậy làm thế nào để quyết định mức độ phức tạp của mô hình? Làm thế nào để nhận biết mô hình của bạn đang bị quá khớp hay thiếu khớp với dữ liệu?

Trong Chương 2, bạn đã sử dụng phương pháp kiểm tra chéo (cross-validation) để ước lượng hiệu suất tổng quát hóa của mô hình. Nếu mô hình hoạt động tốt trên dữ liệu huấn luyện nhưng lại tổng quát hóa kém theo các chỉ số kiểm tra chéo, thì mô hình của bạn đang bị quá khớp. Nếu mô hình hoạt động kém trên cả hai, thì nó đang bị thiếu khớp. Đây là một cách để nhận biết khi nào mô hình quá đơn giản hoặc quá phức tạp.

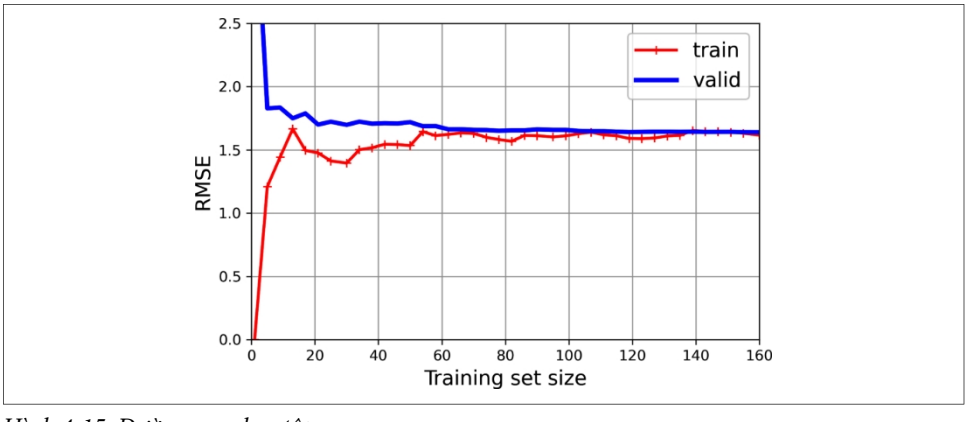
Một cách khác để nhận biết là xem xét các *đường cong học tập*, tức là các đồ thị thể hiện sai số huấn luyện và sai số kiểm định của mô hình theo số lần lặp huấn luyện: chỉ cần đánh giá mô hình tại các khoảng thời gian đều đặn trong quá trình huấn luyện trên cả tập huấn luyện và tập kiểm định, rồi vẽ đồ thị kết quả. Nếu mô hình không thể được huấn luyện theo từng bước (tức là nếu nó không hỗ trợ `partial_fit()` hoặc `warm_start`), thì bạn phải huấn luyện nó nhiều lần trên các tập con ngày càng lớn của tập huấn luyện.

Scikit-Learn có hàm `learning_curve()` hữu ích để hỗ trợ việc này: hàm này huấn luyện và đánh giá mô hình bằng phương pháp kiểm tra chéo. Theo mặc định, hàm này sẽ huấn luyện lại mô hình trên các tập con ngày càng lớn của tập huấn luyện, nhưng nếu mô hình hỗ trợ học tăng dần, bạn có thể đặt `exploit_incremental_learning=True` khi gọi `learning_curve()` và hàm sẽ huấn luyện mô hình theo cách tăng dần thay thế. Hàm này trả về các kích thước tập huấn luyện tại các điểm mà nó đã đánh giá mô hình, cùng với các điểm số huấn luyện và xác thực được đo lường cho từng kích thước và cho từng phần (fold) trong quá trình kiểm tra chéo. Hãy sử dụng hàm này để xem các đường cong học tập của mô hình hồi quy tuyến tính cơ bản (xem Hình 4-15):

```
from sklearn.model_selection import learning_curve

train_sizes, train_scores, valid_scores = learning_curve( LinearRegression(), X,
    y, train_sizes=np.linspace(0.01, 1.0, 40), cv=5,
    scoring="neg_root_mean_squared_error")
train_errors = -train_scores.mean(axis=1)
valid_errors = -valid_scores.mean(axis=1)

plt.plot(train_sizes, train_errors, "r-+", linewidth=2, label="train")
plt.plot(train_sizes, valid_errors, "b-", linewidth=3, label="valid") [...]
# làm đẹp biểu đồ: thêm nhãn, trục tọa độ, lưới và chú thích plt.show()
```



Hình 4-15. Đường cong học tập

Mô hình này đang bị underfitting, tức là nó quá đơn giản so với dữ liệu. Làm sao chúng ta biết được điều này? Hãy xem xét sai số huấn luyện. Khi tập huấn luyện chỉ có một hoặc hai mẫu, mô hình có thể phù hợp hoàn hảo với chúng, đó là lý do tại sao đường cong bắt đầu từ 0. Tuy nhiên, khi các mẫu mới được thêm vào tập huấn luyện, mô hình không thể phù hợp hoàn hảo với dữ liệu huấn luyện nữa, cả vì dữ liệu có nhiễu lẫn vì nó hoàn toàn không tuyến tính. Do đó, sai số trên dữ liệu huấn luyện tăng lên cho đến khi đạt đến mức ổn định, lúc này việc thêm các mẫu mới vào tập huấn luyện không làm cho sai số trung bình tốt hơn hay xấu đi đáng kể. Bây giờ hãy xem xét sai số kiểm định. Khi mô hình được huấn luyện trên rất ít mẫu huấn luyện, nó không thể tổng quát hóa đúng cách, đó là lý do tại sao sai số kiểm định ban đầu khá lớn. Sau đó, khi mô hình được cung cấp thêm các ví dụ huấn luyện, nó học hỏi, và do đó sai số kiểm định từ từ giảm xuống. Tuy nhiên, một lần nữa, một đường thẳng không thể mô hình hóa dữ liệu một cách hiệu quả, nên sai số cuối cùng đạt đến một mức ổn định, rất gần với đường cong kia.

Các đường cong học tập này là đặc trưng của một mô hình bị underfitting. Cả hai đường cong đều đã đạt đến mức ổn định; chúng gần nhau và ở mức khá cao.



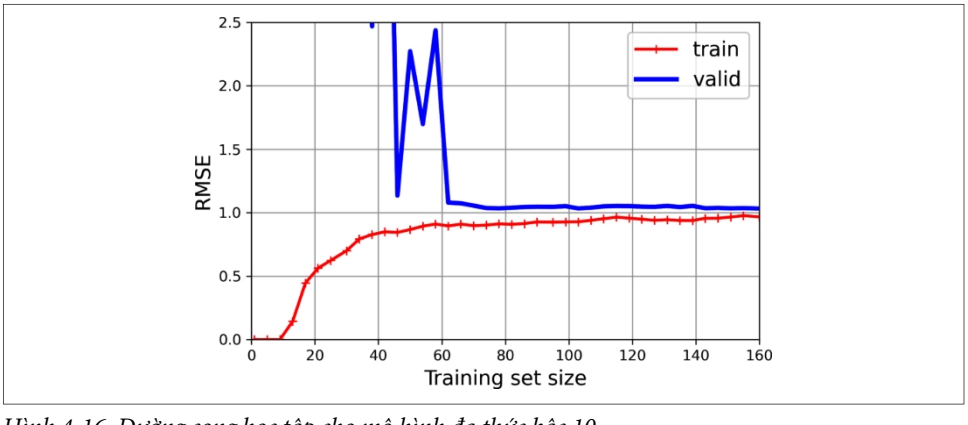
Nếu mô hình của bạn bị hiện tượng underfitting đối với dữ liệu huấn luyện, việc thêm nhiều ví dụ huấn luyện hơn sẽ không giúp ích gì. Bạn cần sử dụng một mô hình tốt hơn hoặc đưa ra các đặc trưng tốt hơn.

Bây giờ, hãy xem xét các đường cong học tập của mô hình đa thức bậc 10 trên cùng một dữ liệu (Hình 4-16):

```
from sklearn.pipeline import make_pipeline

polynomial_regression = make_pipeline(
    PolynomialFeatures(degree=10, include_bias=False),
    LinearRegression())

train_sizes, train_scores, valid_scores = learning_curve( polynomial_regression, X,
    y, train_sizes=np.linspace(0.01, 1.0, 40), cv=5,
    scoring="neg_root_mean_squared_error")
[...] # tương tự như trước
```



Hình 4-16. Đường cong học tập cho mô hình đa thức bậc 10

Các đường cong học tập này trông hơi giống với những đường cong trước đó, nhưng có hai điểm khác biệt rất quan trọng:

- Lỗi trên dữ liệu huấn luyện thấp hơn nhiều so với trước đây.
- Có một khoảng cách giữa các đường cong. Điều này có nghĩa là mô hình hoạt động tốt hơn trên dữ liệu huấn luyện so với dữ liệu xác nhận, đây là dấu hiệu đặc trưng của một mô hình quá khớp. Tuy nhiên, nếu bạn sử dụng tập huấn luyện lớn hơn nhiều, hai đường cong sẽ tiếp tục xích lại gần nhau hơn.

Sự cân bằng giữa sai lệch và phương sai

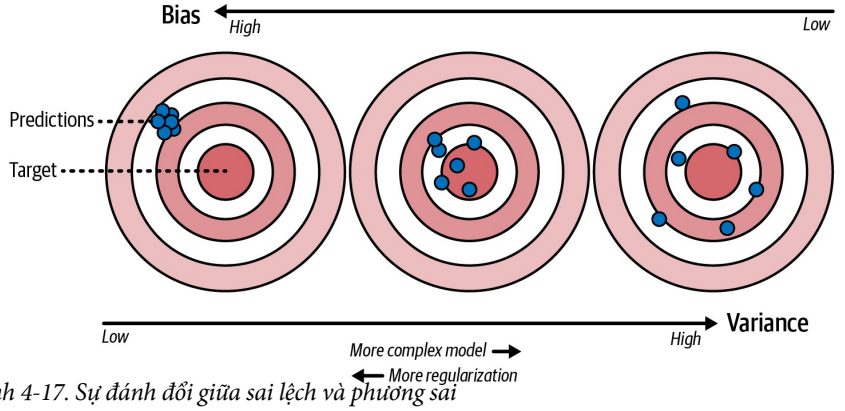
Một kết quả lý thuyết quan trọng trong thống kê và học máy là thực tế rằng sai số tổng quát hóa của một mô hình có thể được biểu diễn dưới dạng tổng của ba loại sai số rất khác nhau:

Sai lệch (Bias)
: Phần này của sai số tổng quát hóa là do các giả định sai lầm, chẳng hạn như giả định rằng dữ liệu là tuyến tính trong khi thực tế nó là bậc hai. Một mô hình có sai lệch cao rất có thể sẽ không phù hợp với dữ liệu huấn luyện.[6]

Độ biến thiên
: Phần này là do mô hình quá nhạy cảm với những biến động nhỏ trong dữ liệu huấn luyện. Một mô hình có nhiều bậc tự do (chẳng hạn như mô hình đa thức bậc cao) có khả năng có phương sai cao và do đó sẽ quá khớp với dữ liệu huấn luyện.

Lỗi không thể giảm thiểu
: Phần này xuất phát từ tính nhiễu của chính dữ liệu. Cách duy nhất để giảm phần sai số này là làm sạch dữ liệu (ví dụ: khắc phục nguồn dữ liệu, như cảm biến hỏng, hoặc phát hiện và loại bỏ các giá trị ngoại lệ).

Việc tăng độ phức tạp của mô hình thường sẽ làm tăng phương sai và giảm sai lệch của mô hình. Ngược lại, việc giảm độ phức tạp của mô hình (hoặc tăng độ chính quy hóa) sẽ làm tăng sai lệch và giảm phương sai của mô hình (xem Hình 4-17). Đây là lý do tại sao nó được gọi là sự cân bằng.



Hình 4-17. Sự đánh đổi giữa sai lệch và phương sai

6 Khái niệm sai lệch này không nên nhầm lẫn với thuật ngữ sai lệch trong các mô hình tuyến tính.



Một cách để cải thiện mô hình quá khớp là cung cấp thêm dữ liệu huấn luyện cho đến khi sai số xác nhận đủ gần với sai số huấn luyện.

Các mô hình tuyến tính được chuẩn hóa

Như bạn đã thấy trong Chương 1 và 2, một cách tốt để giảm hiện tượng quá khớp là thực hiện quy chuẩn hóa mô hình (tức là đặt ra các ràng buộc cho nó): mô hình có càng ít bậc tự do thì càng khó xảy ra hiện tượng quá khớp với dữ liệu. Một cách đơn giản để quy chuẩn hóa mô hình đa thức là giảm số bậc đa thức.

Còn các mô hình tuyến tính thì sao? Chúng ta có thể thường quy chúng không? Bạn có thể thắc mắc tại sao chúng ta lại muốn làm điều đó: chẳng phải các mô hình tuyến tính đã bị ràng buộc đủ rồi sao? Thực ra, hồi quy tuyến tính đưa ra một số giả định, bao gồm việc mối quan hệ thực sự giữa các biến đầu vào và đầu ra là tuyến tính, nhiễu có trung bình bằng không, phương sai hằng số và độc lập với các biến đầu vào, cộng với ma trận đầu vào có hạng đầy đủ, nghĩa là các biến đầu vào không đồng tuyến[7]và số mẫu ít nhất bằng số tham số. Trên thực tế, một số giả định không hoàn toàn đúng. Ví dụ, một số biến đầu vào có thể gần như đồng tuyến, điều này khiến hồi quy tuyến tính trở nên không ổn định về mặt số học, nghĩa là những khác biệt rất nhỏ trong tập huấn luyện có thể gây ra tác động lớn đến mô hình đã được huấn luyện. Quy chuẩn hóa có thể giúp ổn định các mô hình tuyến tính và làm cho chúng chính xác hơn.

Vậy làm thế nào để chúng ta có thể thực hiện quy chuẩn hóa cho một mô hình tuyến tính? Điều này thường được thực hiện bằng cách đặt ra các ràng buộc đối với các trọng số của mô hình. Trong phần này, chúng ta sẽ thảo luận về hồi quy Ridge, hồi quy Lasso và hồi quy Elastic Net, những phương pháp này thực hiện ba cách khác nhau để đạt được mục tiêu đó.

Hồi quy Ridge

Hồi quy Ridge (còn gọi là *chuẩn hóa Tikhonov*) là phiên bản hồi quy tuyến tính được chuẩn hóa: một *hàm chuẩn hóa* bằng $\frac{\alpha}{m}\sum_{i=1}^{n}\theta_i^2$ được thêm vào MSE. Điều này buộc thuật toán học không chỉ phải vừa khít với dữ liệu mà còn phải giữ cho các trọng số của mô hình nhỏ nhất có thể. Hạn chế này làm cho mô hình kém linh hoạt hơn, ngăn không cho nó "kéo giãn" quá mức để vừa khít với mọi điểm dữ liệu: điều này giảm thiểu rủi ro quá khớp. Lưu ý rằng thuật ngữ thường hóa chỉ nên được thêm vào hàm chi phí trong quá trình huấn luyện. Sau khi mô hình được huấn luyện xong, bạn nên sử dụng MSE không thường hóa (hoặc RMSE) để đánh giá hiệu suất của mô hình.

7 Các biến đầu vào được gọi là đồng tuyến khi một biến đầu vào bằng với tổ hợp tuyến tính của một số biến đầu vào khác. Ví dụ, nhiệt độ tính bằng độ C đồng tuyến với nhiệt độ tính bằng độ F.

Siêu tham số α kiểm soát mức độ bạn muốn chuẩn hóa mô hình. Nếu $\alpha = 0$, thì hồi quy Ridge chỉ là hồi quy tuyến tính. Nếu α rất lớn, thì tất cả các trọng số sẽ rất gần với 0 và kết quả là một đường thẳng đi qua giá trị trung bình của dữ liệu. Phương trình 4-9 trình bày hàm chi phí hồi quy Ridge.[8]

Phương trình 4-9. Hàm chi phí hồi quy Ridge

$$J(\theta) = \text{MSE}(\theta) + \frac{\alpha}{m}\sum_{i=1}^{n}\theta_i^2$$

Lưu ý rằng thành phần sai lệch $\theta_0$không được chuẩn hóa (tổng bắt đầu từ $i = 1$, không phải 0). Nếu ta định nghĩa **w** là vectơ trọng số đặc trưng (từ θ_1đến θ_n), thì thuật ngữ chuẩn hóa bằng $\alpha(\|\mathbf{w}\|_2^2)/m$, trong đó$\|\mathbf{w}\|_2$biểu thị chuẩn ℓ_2của vectơ trọng số.[9] Đối với phương pháp gradient descent theo lô, chỉ cần cộng $\mathbf{2\alpha w} / m$ vào phần của vectơ gradient MSE tương ứng với các trọng số đặc trưng, mà không cần cộng thêm gì vào gradient của thành phần bias (xem Phương trình 4-7).

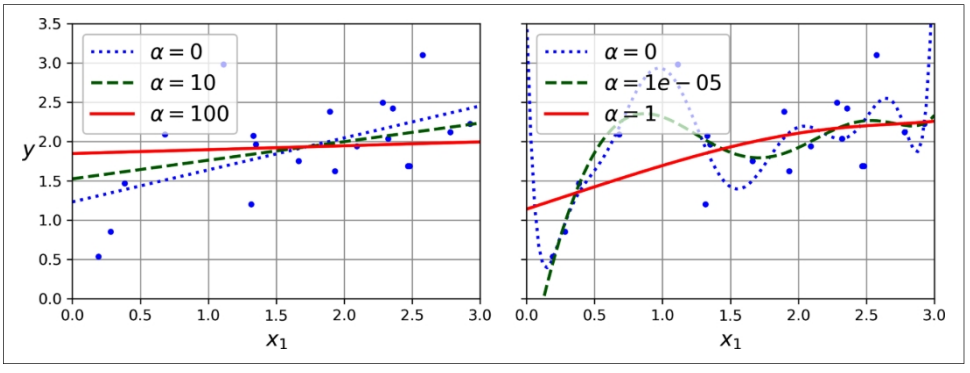


Việc chuẩn hóa dữ liệu (ví dụ: sử dụng `StandardScaler`) trước khi thực hiện hồi quy Ridge là rất quan trọng, vì mô hình này nhạy cảm với quy mô của các đặc trưng đầu vào. Điều này đúng với hầu hết các mô hình được thường hóa.

Hình 4-18 minh họa một số mô hình Ridge được huấn luyện trên dữ liệu tuyến tính có nhiễu mạnh bằng các giá trị α khác nhau. Ở bên trái, các mô hình Ridge thuần túy được sử dụng, dẫn đến các dự đoán tuyến tính. Ở bên phải, dữ liệu trước tiên được mở rộng bằng hàm `PolynomialFeatures(degree=10)`, sau đó được chuẩn hóa bằng `StandardScaler`, và cuối cùng các mô hình Ridge được áp dụng cho các đặc trưng kết quả: đây là hồi quy đa thức với chính quy hóa Ridge. Lưu ý rằng việc tăng α dẫn đến các dự đoán phẳng hơn (tức là ít cực đoan hơn, hợp lý hơn), do đó làm giảm phương sai của mô hình nhưng làm tăng sai lệch của nó.

8 Thông thường, người ta sử dụng ký hiệu $J(\boldsymbol{\theta})$ cho các hàm chi phí không có tên gọi ngắn gọn; tôi sẽ thường xuyên sử dụng ký hiệu này trong phần còn lại của cuốn sách này. Bối cảnh sẽ làm rõ hàm chi phí nào đang được đề cập.

9 Các chuẩn số được thảo luận trong Chương 2.



Hình 4-18. Mô hình tuyến tính (bên trái) và đa thức (bên phải), cả hai đều có các mức độ chính quy hóa ridge khác nhau

Giống như hồi quy tuyến tính, chúng ta có thể thực hiện hồi quy Ridge bằng cách tính phương trình dạng đóng hoặc bằng phương pháp gradient descent. Ưu và nhược điểm là như nhau. Phương trình 4-10 cho thấy giải pháp dạng đóng, trong đó **A** là *ma trận đơn vị* $(n + 1) \times (n + 1)$,[10] ngoại trừ ô trên cùng bên trái có giá trị 0, tương ứng với thành phần sai lệch.

Phương trình 4-10. Giải pháp dạng đóng của hồi quy Ridge

$$\hat{\theta} = \left(X^{\intercal} X + \alpha A \right)^{-1} X^{\intercal} y$$

Dưới đây là cách thực hiện hồi quy Ridge với Scikit-Learn bằng cách sử dụng giải pháp dạng đóng (một biến thể của Phương trình 4-10 sử dụng kỹ thuật phân tích ma trận của André-Louis Cholesky):

```
>>> from sklearn.linear_model import Ridge
>>> ridge_reg = Ridge(alpha=0.1, solver="cholesky")
>>> ridge_reg.fit(X, y)
>>> ridge_reg.predict([[1.5]])
array([1.84414523])
```

Và sử dụng phương pháp gradient ngẫu nhiên:[11]

```
>>> sgd_reg = SGDRegressor(penalty="l2", alpha=0.1 / m, tol=None,
...                        max_iter=1000, eta0=0.01, random_state=42)
...
>>> sgd_reg.fit(X, y.ravel()) # y.ravel() vì hàm fit() yêu cầu dữ liệu mục tiêu dạng 1D
>>> sgd_reg.predict([[1.5]])
array([1.83659707])
```

10 Một ma trận vuông chứa toàn số 0, ngoại trừ các số 1 trên đường chéo chính (từ trên cùng bên trái xuống dưới cùng bên phải).

11 Ngoài ra, bạn có thể sử dụng lớp `Ridge` với bộ giải `"sag"`. GD trung bình ngẫu nhiên là một biến thể của GD ngẫu nhiên. Để biết thêm chi tiết, hãy xem bài thuyết trình "Tối thiểu hóa tổng hữu hạn bằng thuật toán Gradient trung bình ngẫu nhiên" của Mark Schmidt và các cộng sự từ Đại học British Columbia.

Siêu tham số `phạt (penalty)` xác định loại thuật ngữ thường quy sẽ được sử dụng. Việc chỉ định `"l2"` cho biết bạn muốn SGD thêm một thuật ngữ chính quy hóa vào hàm chi phí MSE, bằng `alpha` nhân với bình phương của chuẩn ℓ_2 của vectơ trọng số. Điều này giống như hồi quy Ridge, ngoại trừ việc trong trường hợp này không có phép chia cho *m*; đó là lý do tại sao chúng tôi truyền vào `alpha=0.1 / m`, để có được kết quả tương tự như `Ridge(alpha=0.1)`.



Lớp `RidgeCV` cũng thực hiện hồi quy Ridge, nhưng nó tự động điều chỉnh các siêu tham số bằng cách sử dụng xác nhận chéo. Về cơ bản, nó tương đương với việc sử dụng `GridSearchCV`, nhưng được tối ưu hóa cho hồi quy Ridge và chạy nhanh hơn *nhiều*. Một số ước lượng khác (chủ yếu là tuyến tính) cũng có các biến thể xác nhận chéo hiệu quả, chẳng hạn như `LassoCV` và `ElasticNetCV`.

Hồi quy Lasso

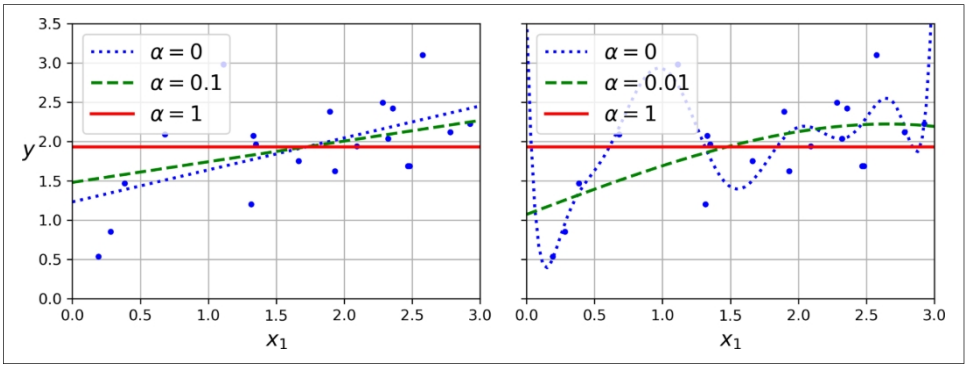
Hồi quy với toán tử thu hẹp và chọn lọc tuyệt đối tối thiểu (thường được gọi tắt là *hồi quy Lasso*) là một biến thể được thường hóa khác của hồi quy tuyến tính: giống như hồi quy Ridge, nó thêm một thuật ngữ thường hóa vào hàm chi phí, nhưng sử dụng chuẩn ℓ_1 của vectơ trọng số thay vì bình phương của chuẩn ℓ_2 (xem Phương trình 4-11). Lưu ý rằng chuẩn ℓ_1 được nhân với 2α, trong khi chuẩn ℓ_2 được nhân với α / m trong hồi quy Ridge. Các hệ số này được chọn để đảm bảo rằng giá trị α tối ưu không phụ thuộc vào kích thước tập huấn luyện: các chuẩn khác nhau dẫn đến các hệ số khác nhau (xem vấn đề #15657 của Scikit-Learn để biết thêm chi tiết).

Phương trình 4-11. Hàm chi phí hồi quy Lasso

$$J(\theta) = \text{MSE}(\theta) + 2\alpha \sum_{i=1}^{n} |\theta_i|$$

Hình 4-19 thể hiện nội dung tương tự như Hình 4-18 nhưng thay thế các mô hình Ridge bằng các mô hình Lasso và sử dụng các giá trị α khác nhau.

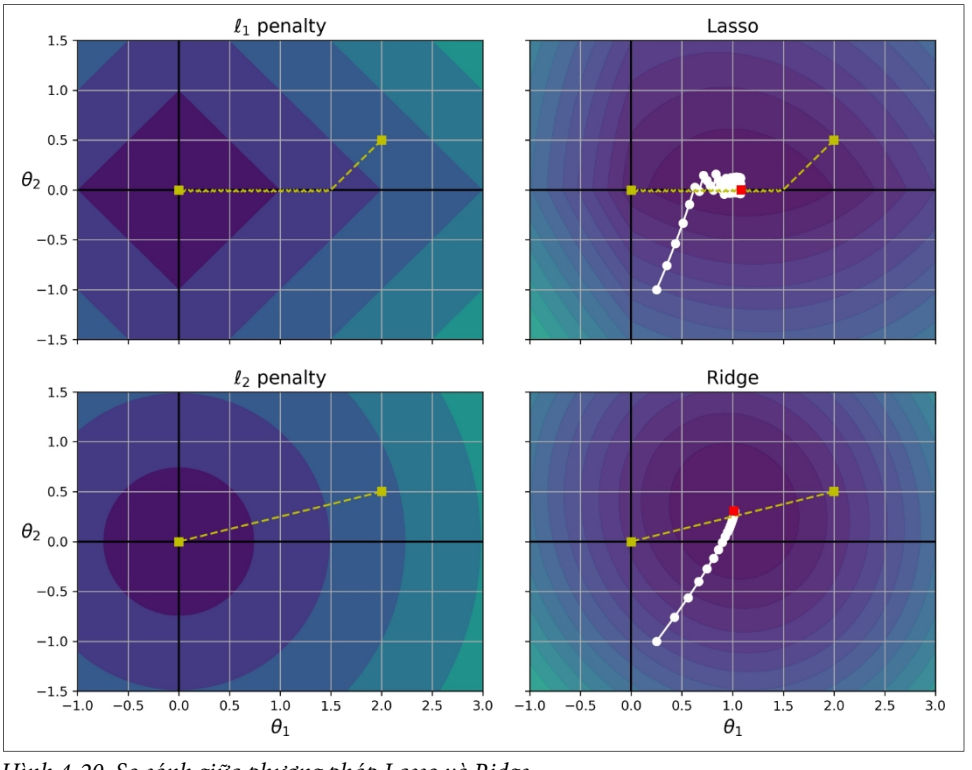
Một đặc điểm quan trọng của hồi quy Lasso là nó có xu hướng loại bỏ các hệ số của các đặc trưng ít quan trọng nhất (tức là đặt chúng bằng không). Ví dụ, đường đứt nét trong biểu đồ bên phải của Hình 4-19 (với $\alpha = 0{,}01$) trông gần giống hình lập phương: tất cả các hệ số của các đặc trưng đa thức bậc cao đều bằng không. Nói cách khác, hồi quy Lasso tự động thực hiện việc lựa chọn đặc trưng và đưa ra một *mô hình thưa thớt* với rất ít trọng số đặc trưng khác không. Tất nhiên, điều này đi kèm với sự đánh đổi: nếu bạn tăng α quá nhiều, mô hình sẽ trở nên rất thưa thớt, nhưng hiệu suất của nó sẽ giảm mạnh.



Hình 4-19. Các mô hình tuyến tính (bên trái) và đa thức (bên phải), cả hai đều sử dụng các mức độ khác nhau của quy tắc hóa Lasso

Bạn có thể hiểu được lý do tại sao chuẩn ℓ_1tạo ra tính thưa thớt bằng cách xem Hình 4-20: các trục biểu diễn hai tham số mô hình, và các đường viền nền biểu diễn các hàm mất mát khác nhau. Trong biểu đồ góc trên bên trái, các đường viền biểu diễn hàm mất mát $\ell_1(|\theta_1| + |\theta_2|)$, giá trị này giảm tuyến tính khi bạn tiến gần hơn đến bất kỳ trục nào. Ví dụ, nếu bạn khởi tạo các tham số mô hình là $\theta_1 = 2$ và $\theta_2 = 0{,}5$, việc áp dụng thuật toán gradient descent sẽ làm giảm cả hai tham số một cách đồng đều (như được thể hiện bởi đường đứt nét màu vàng); do đó, θ_2 sẽ đạt 0 trước (vì ban đầu nó đã gần 0 hơn). Sau đó, thuật toán gradient descent sẽ lăn xuống rãnh cho đến khi đạt đến $\theta_1 = 0$ (với một chút dao động, vì đạo hàm của $\ell_1$không bao giờ gần bằng 0: chúng luôn là –1 hoặc 1 đối với mỗi tham số). Trong đồ thị phía trên bên phải, các đường đồng mức biểu diễn hàm chi phí của hồi quy Lasso (tức là hàm chi phí MSE cộng với hàm mất mát ℓ_1). Các vòng tròn trắng nhỏ cho thấy đường đi mà thuật toán gradient descent thực hiện để tối ưu hóa một số tham số mô hình được khởi tạo xung quanh $\theta_1 = 0{,}25$ và $\theta_2 = -1$: hãy lưu ý một lần nữa cách đường đi này nhanh chóng đạt đến $\theta_2 = 0$, sau đó lăn xuống rãnh và cuối cùng dao động quanh điểm cực đại toàn cục (được biểu diễn bằng hình vuông màu đỏ). Nếu tăng α, điểm cực đại toàn cục sẽ dịch chuyển sang trái dọc theo đường đứt nét màu vàng, trong khi nếu giảm α, điểm cực đại toàn cục sẽ dịch chuyển sang phải (trong ví dụ này, các tham số tối ưu cho MSE không được thường hóa là $\theta_1 = 2$ và $\theta_2 = 0{,}5$).

Hai đồ thị dưới cùng thể hiện điều tương tự nhưng sử dụng $_{\text{hàm}}$phạt $\ell_2$thay thế. Trong đồ thị dưới cùng bên trái, bạn có thể thấy rằng hàm phạt $\ell_2$giảm dần khi chúng ta tiến gần đến gốc tọa độ, do đó thuật toán gradient descent chỉ đi theo một đường thẳng hướng về điểm đó. Trong đồ thị dưới cùng bên phải, các đường đồng mức biểu diễn hàm chi phí của hồi quy Ridge (tức là hàm chi phí MSE cộng với hàm mất mát ℓ_2). Như bạn có thể thấy, độ dốc giảm dần khi các tham số tiến gần đến điểm tối ưu toàn cục, do đó phương pháp gradient descent tự nhiên sẽ chậm lại. Điều này hạn chế sự dao động, giúp hồi quy ridge hội tụ nhanh hơn so với hồi quy lasso. Cũng cần lưu ý rằng các tham số tối ưu (được biểu diễn bằng hình vuông màu đỏ) ngày càng tiến gần đến gốc tọa độ khi bạn tăng α, nhưng chúng không bao giờ bị loại bỏ hoàn toàn.



Hình 4-20. So sánh giữa phương pháp Lasso và Ridge



Để ngăn chặn quá trình giảm dốc gradient dao động quanh điểm cực đại ở giai đoạn cuối khi sử dụng hồi quy Lasso, bạn cần giảm dần tốc độ học trong quá trình huấn luyện. Quá trình vẫn sẽ dao động quanh điểm cực đại, nhưng các bước sẽ ngày càng nhỏ hơn, do đó nó sẽ hội tụ.

Hàm chi phí Lasso không khả vi tại các *điểm* $\theta_i = 0$ (cho $i = 1, 2, \cdots, n$), nhưng phương pháp gradient descent vẫn hoạt động nếu bạn sử dụng *vectơ phụ đạo* $\mathbf{g}$[12] thay thế khi bất kỳ θ_i nào bằng 0. Phương trình 4-12 trình bày phương trình vectơ phụ đạo mà bạn có thể sử dụng cho phương pháp gradient descent với hàm chi phí Lasso.

12 Bạn có thể coi vectơ phụ đạo tại một điểm không khả vi như một vectơ trung gian giữa các vectơ đạo hàm xung quanh điểm đó.

Phương trình 4-12. Vectơ phụ đạo trong hồi quy Lasso

$$g(\theta) = \nabla_\theta \text{MSE}(\theta) + 2\alpha \begin{pmatrix} \text{dấu}(\theta_1) \\ \text{dấu}(\theta_2) \\ \vdots \\ \text{dấu}(\theta_n) \end{pmatrix} \quad \text{trong đó dấu}(\theta_i) = \begin{cases} -1 \text{ nếu } \theta_i < 0 \\ 0 \text{ nếu } \theta_i = 0 \\ +1 \text{ nếu } \theta_i > 0 \end{cases}$$

Dưới đây là một ví dụ nhỏ về Scikit-Learn sử dụng lớp `Lasso`:

```
>>> from sklearn.linear_model import Lasso
>>> lasso_reg = Lasso(alpha=0.1)
>>> lasso_reg.fit(X, y)
>>> lasso_reg.predict([[1.5]])
array([1.87550211])
```

Lưu ý rằng bạn cũng có thể sử dụng `SGDRegressor(penalty="l1", alpha=0.1)` thay thế.

Hồi quy Elastic Net

Hồi quy Elastic Net là giải pháp trung gian giữa hồi quy Ridge và hồi quy Lasso. Hàm chuẩn hóa là tổng có trọng số của cả hai hàm chuẩn hóa Ridge và Lasso, và bạn có thể điều chỉnh tỷ lệ trộn *r*. Khi *r* = 0, Elastic Net tương đương với hồi quy Ridge, và khi *r* = 1, nó tương đương với hồi quy Lasso (Phương trình 4-13).

Phương trình 4-13. Hàm chi phí Elastic Net

$$J(\theta) = \text{MSE}(\theta) + r\alpha \sum_{i=1}^{n} |\theta_i| + \frac{1-r}{2}\alpha \sum_{i=1}^{n} \theta_i^2$$

Vậy khi nào bạn nên sử dụng hồi quy elastic net, hay ridge, lasso, hoặc hồi quy tuyến tính đơn thuần (tức là không có bất kỳ hình thức thường quy nào)? Hầu như luôn luôn tốt hơn khi có ít nhất một chút thường quy, vì vậy nói chung bạn nên tránh hồi quy tuyến tính đơn thuần. Ridge là một lựa chọn mặc định tốt, nhưng nếu bạn nghi ngờ rằng chỉ có một vài đặc trưng hữu ích, bạn nên ưu tiên lasso hoặc elastic net vì chúng có xu hướng giảm trọng số của các đặc trưng vô dụng xuống 0, như đã thảo luận trước đó. Nói chung, elastic net được ưa chuộng hơn lasso vì lasso có thể hoạt động không ổn định khi số lượng đặc trưng lớn hơn số lượng mẫu huấn luyện hoặc khi một số đặc trưng có mối tương quan mạnh.

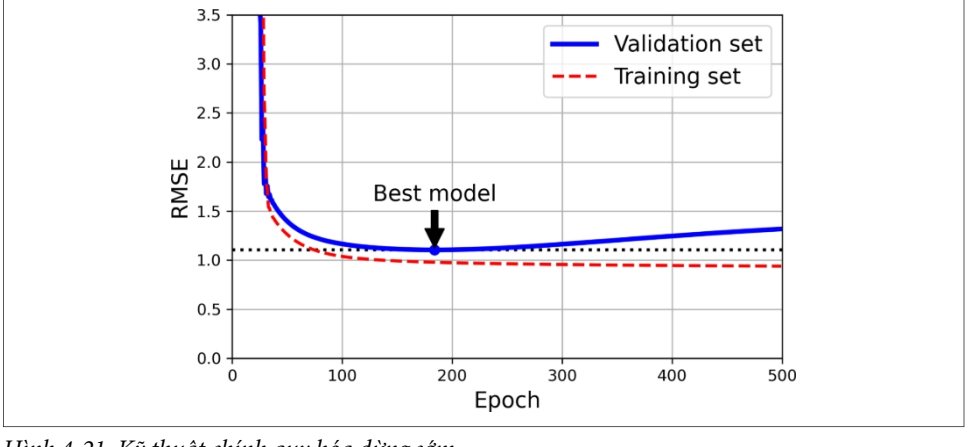
Dưới đây là một ví dụ ngắn sử dụng `ElasticNet` của Scikit-Learn (`l1_ratio` tương ứng với tỷ lệ trộn *r*):

```
>>> from sklearn.linear_model import ElasticNet
>>> elastic_net = ElasticNet(alpha=0.1, l1_ratio=0.5)
>>> elastic_net.fit(X, y)
```

```
>>> elastic_net.predict([[1.5]])
array([1.8645014])
```

Dừng sớm

Một cách khác để chuẩn hóa các thuật toán học lặp như gradient descent là dừng huấn luyện ngay khi sai số xác nhận đạt mức tối thiểu. Kỹ thuật phổ biến này được gọi là *dừng sớm*. Hình 4-21 cho thấy một mô hình phức tạp (trong trường hợp này là mô hình hồi quy đa thức bậc cao) đang được huấn luyện bằng phương pháp gradient descent theo lô trên tập dữ liệu bậc hai mà chúng ta đã sử dụng trước đó. Khi các epoch trôi qua, thuật toán học hỏi và sai số dự đoán (RMSE) trên tập huấn luyện giảm xuống, cùng với sai số dự đoán trên tập kiểm định. Tuy nhiên, sau một thời gian, sai số kiểm định ngừng giảm và bắt đầu tăng trở lại. Điều này cho thấy mô hình đã bắt đầu bị quá khớp với dữ liệu huấn luyện. Với phương pháp dừng sớm, bạn chỉ cần dừng huấn luyện ngay khi sai số xác nhận đạt mức tối thiểu. Đây là một kỹ thuật chuẩn hóa đơn giản và hiệu quả đến mức Geoffrey Hinton đã gọi nó là "bữa trưa miễn phí tuyệt vời".[13] Tuy nhiên, đôi khi sai số xác nhận lại giảm xuống sau một thời gian: hiện tượng này được gọi là "*double descent*". Hiện tượng này khá phổ biến với các mạng nơ-ron lớn và là một lĩnh vực nghiên cứu đang được quan tâm.



Hình 4-21. Kỹ thuật chính quy hóa dừng sớm



Với phương pháp gradient descent ngẫu nhiên và mini-batch, các đường cong không được mượt mà lắm, và có thể khó xác định liệu bạn đã đạt đến điểm cực tiểu hay chưa. Một giải pháp là chỉ dừng lại sau khi sai số xác thực đã duy trì ở mức cao hơn điểm cực tiểu trong một khoảng thời gian nhất định (khi bạn chắc chắn rằng mô hình sẽ không thể cải thiện hơn nữa), sau đó khôi phục các tham số mô hình về điểm mà tại đó sai số xác thực đạt cực tiểu.

13 Trang chiếu số 63 của Hướng dẫn Học sâu tại NeurIPS 2015.

Dưới đây là cách triển khai cơ bản của phương pháp dừng sớm:

```
from copy import deepcopy
from sklearn.metrics import root_mean_squared_error
from sklearn.preprocessing import StandardScaler

X_train, y_train, X_valid, y_valid = [...] # chia tập dữ liệu bậc hai

preprocessing = make_pipeline(PolynomialFeatures(degree=90, include_bias=False),
                              StandardScaler())
X_train_prep = preprocessing.fit_transform(X_train)
X_valid_prep = preprocessing.transform(X_valid)
sgd_reg = SGDRegressor(penalty=None, eta0=0.002, random_state=42) n_epochs =
500
best_valid_rmse = float('inf')

for epoch in range(n_epochs):
    sgd_reg.partial_fit(X_train_prep, y_train)
    y_valid_predict = sgd_reg.predict(X_valid_prep)
    val_error = root_mean_squared_error(y_valid, y_valid_predict)
    if val_error < best_valid_rmse:
        best_valid_rmse = val_error
        best_model = deepcopy(sgd_reg)
```

Mã này trước tiên thêm các đặc trưng đa thức và chuẩn hóa tất cả các đặc trưng đầu vào, cho cả tập huấn luyện và tập kiểm định (mã giả định rằng bạn đã chia tập huấn luyện ban đầu thành một tập huấn luyện nhỏ hơn và một tập kiểm định). Sau đó, nó tạo mô hình `SGDRegressor` không có chính quy hóa và với tốc độ học nhỏ. Trong vòng lặp huấn luyện, nó gọi hàm `partial_fit()` thay vì `fit()`, để thực hiện học tăng dần. Tại mỗi epoch, nó đo lường RMSE trên tập kiểm định. Nếu giá trị này thấp hơn giá trị RMSE thấp nhất đã ghi nhận cho đến nay, nó sẽ lưu một bản sao của mô hình vào biến `best_model`. Cách triển khai này thực tế không dừng quá trình huấn luyện, nhưng cho phép bạn quay lại sử dụng mô hình tốt nhất sau khi huấn luyện xong. Lưu ý rằng mô hình được sao chép bằng hàm `` `copy.deepcopy()` ``, vì hàm này sao chép cả các siêu tham số *và* các tham số đã học của mô hình. Ngược lại, hàm `` `sklearn.base.clone()` `` chỉ sao chép các siêu tham số của mô hình.

Hồi quy logistic

Như đã thảo luận trong Chương 1, một số thuật toán hồi quy có thể được sử dụng cho phân loại (và ngược lại). *Hồi quy logistic* (còn gọi là *hồi quy logit*) thường được sử dụng để ước tính xác suất một mẫu thuộc về một lớp cụ thể (ví dụ: xác suất email này là spam là bao nhiêu?). Nếu xác suất ước tính lớn hơn một ngưỡng nhất định (thường là 50%), thì mô hình sẽ dự đoán rằng trường hợp đó thuộc về lớp đó (gọi là *lớp dương*, được gắn nhãn "1"), và ngược lại, mô hình sẽ dự đoán rằng trường hợp đó không thuộc về *lớp* đó (tức là thuộc về *lớp âm*, được gắn nhãn "0"). Điều này khiến nó trở thành một bộ phân loại nhị phân.

Ước tính xác suất

Vậy hồi quy logistic hoạt động như thế nào? Giống như mô hình hồi quy tuyến tính, mô hình hồi quy logistic tính toán tổng có trọng số của các đặc trưng đầu vào (cộng với một hằng số dịch chuyển), nhưng thay vì đưa ra kết quả trực tiếp như mô hình hồi quy tuyến tính, nó đưa ra hàm *logistic* của kết quả này (xem Phương trình 4-14).

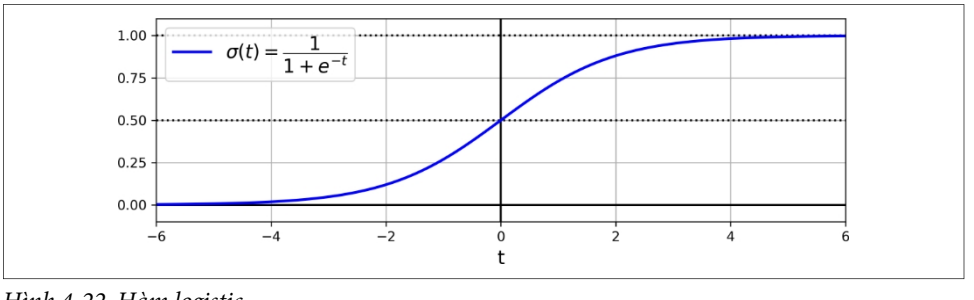
Phương trình 4-14. Xác suất ước lượng của mô hình hồi quy logistic (dạng vectơ)

$$\hat{p} = h_{\theta}(\mathrm{x}) = \sigma(\theta^{\mathsf{T}} x)$$

Hàm logistic — được ký hiệu là $\sigma(\cdot)$ — là một *hàm sigmoid* (tức là có dạng chữ S) cho ra kết quả là một số nằm trong khoảng từ 0 đến 1. Hàm này được định nghĩa như trong Phương trình 4-15 và Hình 4-22.

Phương trình 4-15. Hàm logistic

$$\sigma(t) = \frac{1}{1 + \exp(-t)}$$



Hình 4-22. Hàm logistic

Khi mô hình hồi quy logistic đã ước tính xác suất $p = h_{\theta}(\mathbf{x})$ rằng một trường hợp $\mathbf{x}$ thuộc lớp dương, nó có thể dễ dàng đưa ra dự đoán $\hat{y}$ (xem Phương trình 4-16).

Phương trình 4-16. Dự đoán của mô hình hồi quy logistic sử dụng ngưỡng xác suất 50%

$$\hat{y} = \begin{cases} 0 \text{ nếu } \hat{p} < 0{,}5 \\ 1 \text{ nếu } \hat{p} \geq 0{,}5 \end{cases}$$

Lưu ý rằng $\sigma(t) < 0{,}5$ khi $t < 0$, và $\sigma(t) \geq 0{,}5$ khi $t \geq 0$, do đó mô hình hồi quy logistic sử dụng ngưỡng xác suất mặc định 50% sẽ dự đoán 1 nếu $\boldsymbol{\theta}^T\mathbf{x}$ thuộc lớp dương và 0 nếu thuộc lớp âm.



Điểm t thường được gọi là *logit*. Tên gọi này xuất phát từ thực tế rằng hàm logit, được định nghĩa là $\text{logit}(p) = \log(p / (1 - p))$, là hàm nghịch đảo của hàm logistic. Thật vậy, nếu bạn tính logit của xác suất ước tính p, bạn sẽ thấy rằng kết quả là t. Logit còn được gọi là *log-odds*, vì nó là logarit của tỷ lệ giữa xác suất ước tính cho lớp dương và xác suất ước tính cho lớp âm.

Huấn luyện và hàm chi phí

Bây giờ bạn đã biết cách mô hình hồi quy logistic ước lượng xác suất và đưa ra dự đoán. Nhưng mô hình được huấn luyện như thế nào? Mục tiêu của việc huấn luyện là thiết lập vectơ tham số $\boldsymbol{\theta}$ sao cho mô hình ước lượng xác suất cao cho các trường hợp dương ($y = 1$) và xác suất thấp cho các trường hợp âm ($y = 0$). Ý tưởng này được thể hiện qua hàm chi phí được trình bày trong Phương trình 4-17 cho một trường hợp huấn luyện đơn lẻ $\mathbf{x}$.

Phương trình 4-17. Hàm chi phí của một trường hợp huấn luyện đơn lẻ

$$c(\theta) = \begin{cases} -\log(\hat{p}) & \text{nếu } y = 1 \text{ thì} \\ -\log(1 - \hat{p}) & \text{nếu } y = 0 \end{cases}$$

Hàm chi phí này có ý nghĩa vì $-\log(t)$ tăng rất lớn khi t tiến tới 0, do đó chi phí sẽ lớn nếu mô hình ước lượng xác suất gần 0 cho một trường hợp dương, và nó cũng sẽ lớn nếu mô hình ước lượng xác suất gần 1 cho một trường hợp âm. Mặt khác, $-\log(t)$ gần bằng 0 khi t gần bằng 1, do đó chi phí sẽ gần bằng 0 nếu xác suất ước tính gần bằng 0 đối với một trường hợp âm hoặc gần bằng 1 đối với một trường hợp dương, và đó chính xác là điều chúng ta mong muốn.

Hàm chi phí trên toàn bộ tập huấn luyện là chi phí trung bình trên tất cả các trường hợp huấn luyện. Nó có thể được viết dưới dạng một biểu thức duy nhất gọi là *log loss*, được thể hiện trong Phương trình 4-18.

Phương trình 4-18. Hàm chi phí hồi quy logistic (log loss)

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log\left(\hat{p}^{(i)}\right) + \left(1 - y^{(i)}\right)\log\left(1 - \hat{p}^{(i)}\right)\right]$$



Log loss không phải là một khái niệm được đưa ra một cách tùy tiện. Có thể chứng minh toán học (sử dụng suy luận Bayes) rằng việc tối thiểu hóa hàm mất mát này sẽ dẫn đến mô hình có khả năng tối ưu *cao nhất*, giả sử các mẫu dữ liệu tuân theo phân phối Gaussian xung quanh giá trị trung bình của lớp của chúng. Khi bạn sử dụng log loss, đây chính là giả định ngầm mà bạn đang đưa ra. Càng sai lệch so với giả định này, mô hình sẽ càng bị sai lệch. Tương tự, khi chúng ta sử dụng MSE để huấn luyện các mô hình hồi quy tuyến tính, chúng ta đang ngầm giả định rằng dữ liệu là hoàn toàn tuyến tính, cộng với một ít nhiễu Gaussian. Vì vậy, nếu dữ liệu không tuyến tính (ví dụ: nếu nó là bậc hai) hoặc nếu nhiễu không phải là Gaussian (ví dụ: nếu các giá trị ngoại lai không hiếm theo hàm mũ), thì mô hình sẽ bị sai lệch.

Tin xấu là hiện chưa có phương trình dạng đóng nào để tính giá trị của **θ** làm tối thiểu hàm chi phí này (không có phương trình tương đương với phương trình chuẩn). Tuy nhiên, tin tốt là hàm chi phí này là lồi, do đó phương pháp gradient descent (hoặc bất kỳ thuật toán tối ưu hóa nào khác) đều đảm bảo tìm được cực tiểu toàn cục (nếu tốc độ học không quá lớn và bạn chờ đủ lâu). Các đạo hàm riêng của hàm chi phí đối với tham số mô hình$^{\text{thứ}\ j}\ \theta_j$được cho bởi Phương trình 4-19.

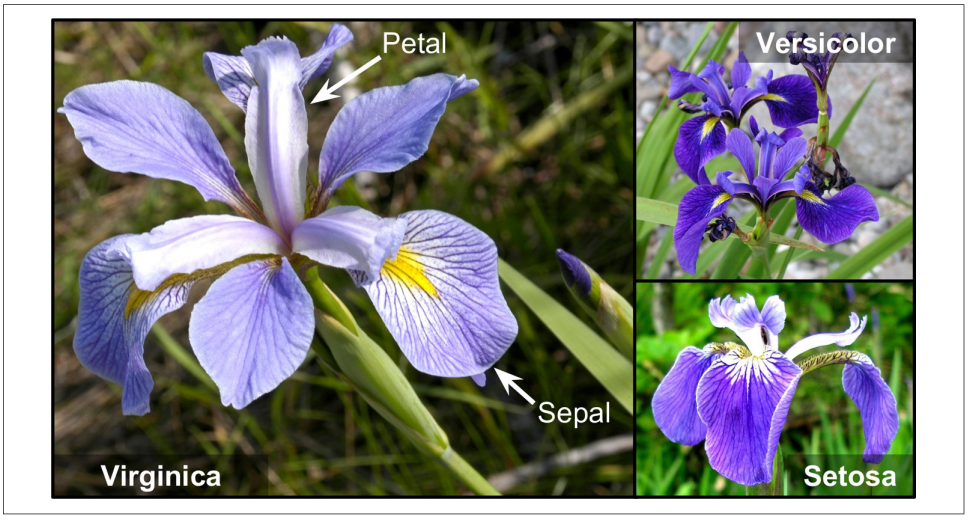
Phương trình 4-19. Đạo hàm riêng của hàm chi phí logistic

$$\frac{\partial}{\partial \theta_j} \mathrm{J}(\theta) = \frac{1}{m} \sum_{i=1}^{m} \left(\sigma\left(\theta^{\mathsf{T}} x^{(i)}\right) - y^{(i)} \right) x_j^{(i)}$$

Phương trình này trông rất giống Phương trình 4-6: đối với mỗi trường hợp, nó tính toán sai số dự đoán và nhân nó với giá trị đặc trưng$^{\text{thứ}\ j}$, sau đó tính trung bình trên tất cả các trường hợp huấn luyện. Khi bạn đã có vectơ gradient chứa tất cả các đạo hàm riêng, bạn có thể sử dụng nó trong thuật toán gradient descent theo lô. Vậy là xong: bây giờ bạn đã biết cách huấn luyện mô hình hồi quy logistic. Đối với phương pháp gradient ngẫu nhiên (stochastic GD), bạn sẽ xử lý từng trường hợp một, còn đối với phương pháp gradient theo lô nhỏ (mini-batch GD), bạn sẽ xử lý từng lô nhỏ một.

Ranh giới quyết định

Chúng ta có thể sử dụng tập dữ liệu hoa diên vĩ để minh họa hồi quy logistic. Đây là một tập dữ liệu nổi tiếng chứa chiều dài và chiều rộng của lá đài và cánh hoa của 150 bông hoa diên vĩ thuộc ba loài khác nhau: *Iris setosa*, *Iris versicolor* và *Iris virginica* (xem Hình 4-23).



Hình 4-23. Hoa của ba loài cây hoa diên vĩ[14]

Hãy thử xây dựng một mô hình phân loại để xác định loài *Iris virginica* chỉ dựa trên đặc trưng chiều rộng cánh hoa. Bước đầu tiên là tải dữ liệu và xem qua nhanh:

```
>>> from sklearn.datasets import load_iris
>>> iris = load_iris(as_frame=True)
>>> list(iris)
['data', 'target', 'frame', 'target_names', 'DESCR', 'feature_names',
 'filename', 'data_module']
>>> iris.data.head(3)
   chiều dài lá đài (cm)  chiều rộng lá đài (cm)  chiều dài cánh hoa (cm)  chiều rộng cánh hoa (cm)
0                    5,1                     3,5                      1,4                      0,2
1                    4,9                     3,0                      1,4                      0,2
2                    4,7                     3,2                      1,3                      0,2
>>> iris.target.head(3) # lưu ý rằng các bản sao không được trộn ngẫu nhiên
0    0
1    0
2    0
Tên: target, kiểu dữ liệu: int64
>>> iris.target_names
array(['setosa', 'versicolor', 'virginica'], dtype='<U10')
```

Tiếp theo, chúng ta sẽ chia dữ liệu và huấn luyện mô hình hồi quy logistic trên tập huấn luyện:

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
```

14 Các bức ảnh được trích dẫn từ các trang Wikipedia tương ứng. Ảnh loài *Iris virginica* do Frank Mayfield chụp (Giấy phép Creative Commons BY-SA 2.0), ảnh loài *Iris versicolor* do D. Gordon E. Robertson chụp (Giấy phép Creative Commons BY-SA 3.0), ảnh loài *Iris setosa* thuộc phạm vi công cộng.

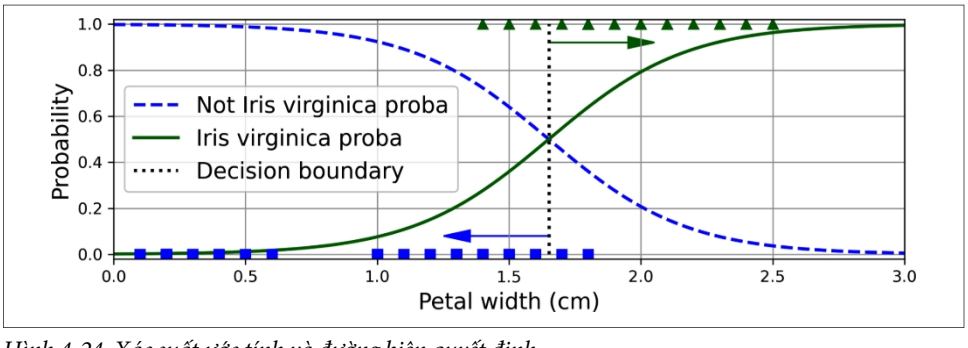
```
X = iris.data[["chiều rộng cánh hoa (cm)"]].values
y = iris.target_names[iris.target] == 'virginica'
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

log_reg = LogisticRegression(random_state=42)
log_reg.fit(X_train, y_train)
```

Hãy xem xét các xác suất ước tính của mô hình đối với các bông hoa có chiều rộng cánh hoa dao động từ 0 cm đến 3 cm (Hình 4-24):[15]

```
X_new = np.linspace(0, 3, 1000).reshape(-1, 1) # định dạng lại để có được một vectơ cột
y_proba = log_reg.predict_proba(X_new) decision_boundary
= X_new[y_proba[:, 1] >= 0.5][0, 0]

plt.plot(X_new, y_proba[:, 0], "b--", linewidth=2,
         label="Không phải Iris virginica proba")
plt.plot(X_new, y_proba[:, 1], "g-", linewidth=2, label="Iris virginica proba")
plt.plot([decision_boundary, decision_boundary], [0, 1], "k:", linewidth=2,
         label="Ranh giới quyết định")
[...] # làm đẹp hình vẽ: thêm lưới, nhãn, trục tọa độ, chú thích, mũi tên và các mẫu
plt.show()
```



Hình 4-24. Xác suất ước tính và đường biên quyết định

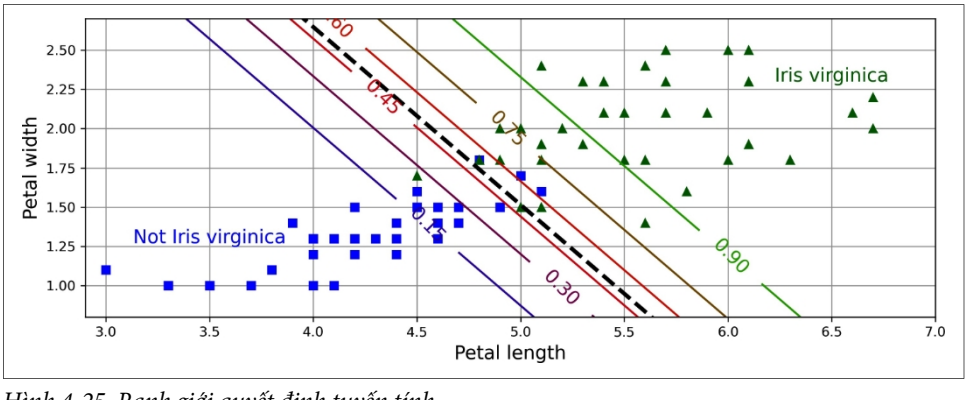
Chiều rộng cánh hoa của hoa *Iris virginica* (được biểu diễn bằng các hình tam giác) dao động từ 1,4 cm đến 2,5 cm, trong khi các loài hoa iris khác (được biểu diễn bằng các hình vuông) thường có chiều rộng cánh hoa nhỏ hơn, dao động từ 0,1 cm đến 1,8 cm. Lưu ý rằng có một chút chồng chéo. Trên khoảng 2 cm, mô hình phân loại rất tự tin rằng bông hoa là *Iris virginica* (nó trả về xác suất cao cho lớp đó), trong khi dưới 1 cm, nó rất tự tin rằng đó không phải là *Iris virginica* (xác suất cao cho lớp "Không phải Iris virginica"). Trong khoảng giữa hai giá trị cực trị này, bộ phân loại không chắc chắn. Tuy nhiên, nếu bạn yêu cầu nó dự đoán lớp (sử dụng phương thức `predict()` thay vì phương thức `predict_proba()`), nó sẽ trả về lớp nào có xác suất cao nhất. Do đó, có một *ranh giới quyết định* ở khoảng 1,6 cm, nơi cả hai xác suất đều bằng 50%: nếu chiều rộng cánh hoa

15 Hàm `reshape()` của NumPy cho phép một chiều có giá trị là –1, điều này có nghĩa là "tự động": giá trị này được suy ra từ độ dài của mảng và các chiều còn lại.

nếu chiều rộng lớn hơn 1,6 cm, bộ phân loại sẽ dự đoán rằng bông hoa là loài *Iris virginica*, còn nếu không thì nó sẽ dự đoán rằng đó không phải là loài này (ngay cả khi độ tin cậy không cao):

```
>>> decision_boundary
np.float64(1.6516516516516517)
>>> log_reg.predict([[1.7], [1.5]])
array([ True, False])
```

Hình 4-25 hiển thị cùng bộ dữ liệu, nhưng lần này trình bày hai đặc trưng: chiều rộng và chiều dài cánh hoa. Sau khi được huấn luyện, mô hình hồi quy logistic có thể, dựa trên hai đặc trưng này, ước tính xác suất rằng một bông hoa mới là *Iris virginica*. Đường đứt nét biểu thị các điểm mà mô hình ước tính xác suất là 50%: đây chính là đường biên quyết định của mô hình. Lưu ý rằng đây là một ranh giới tuyến tính.[16] Mỗi đường song song biểu thị các điểm mà mô hình đưa ra một xác suất cụ thể, từ 15% (góc dưới bên trái) đến 90% (góc trên bên phải). Theo mô hình, tất cả các bông hoa nằm ngoài đường ở góc trên bên phải đều có xác suất trên 90% là *loài Iris virginica*.



Hình 4-25. Ranh giới quyết định tuyến tính



Siêu tham số điều khiển mức độ chính quy hóa của mô hình `LogisticRegression` trong Scikit-Learn không phải là `alpha` (như trong các mô hình tuyến tính khác), mà là giá trị nghịch đảo của nó: `C`. Giá trị của `C` càng cao, mức độ chính quy hóa của mô hình *càng thấp*.

Cũng giống như các mô hình tuyến tính khác, các mô hình hồi quy logistic có thể được thường quy hóa bằng cách sử dụng các hình phạt ℓ_1 hoặc ℓ_2. Scikit-Learn thực tế đã thêm hình phạt ℓ_2 theo mặc định.

16 Đó là tập hợp các điểm **x** sao cho $\theta_0 + \theta_1 x_1 + \theta_2 x_2 = 0$, định nghĩa một đường thẳng.

Hồi quy Softmax

Mô hình hồi quy logistic có thể được tổng quát hóa để hỗ trợ nhiều lớp trực tiếp, mà không cần phải huấn luyện và kết hợp nhiều bộ phân loại nhị phân (như đã thảo luận trong Chương 3). Điều này được gọi là *hồi quy softmax*, hay *hồi quy logistic đa thức*.

Ý tưởng rất đơn giản: khi được cung cấp một mẫu **x**, mô hình hồi quy softmax trước tiên tính toán điểm số $s_k(\mathbf{x})$ cho mỗi lớp *k*, sau đó ước lượng xác suất của mỗi lớp bằng cách áp dụng *hàm softmax* (còn gọi là *hàm mũ chuẩn hóa*) vào các điểm số đó. Phương trình để tính toán $s_k(\mathbf{x})$ có lẽ trông quen thuộc, vì nó giống hệt phương trình dự đoán hồi quy tuyến tính (xem Phương trình 4-20).

Phương trình 4-20. Điểm số softmax cho lớp k

$$s_k(\mathbf{x}) = \left(\boldsymbol{\theta}^{(k)}\right)^T \mathbf{x}$$

Lưu ý rằng mỗi lớp có vectơ tham số riêng $\boldsymbol{\theta}^{(k)}$. Tất cả các vectơ này thường được lưu trữ dưới dạng các hàng trong *ma trận tham số* $\boldsymbol{\Theta}$.

Sau khi tính toán điểm số của từng lớp cho mẫu **x**, bạn có thể ước tính xác suất p_k rằng mẫu đó thuộc lớp *k* bằng cách đưa các điểm số này vào hàm softmax (Phương trình 4-21). Hàm này tính hàm mũ của mỗi điểm số, sau đó chuẩn hóa chúng (chia cho tổng của tất cả các hàm mũ). Các điểm số này thường được gọi là logit hoặc log-odds (mặc dù thực tế chúng là log-odds chưa được chuẩn hóa).

Phương trình 4-21. Hàm softmax

$$\hat{p}_k = \sigma(\mathbf{s}(\mathbf{x}))_k = \frac{\exp\left(s_k(\mathbf{x})\right)}{\sum_{j=1}^{K} \exp\left(s_j(\mathbf{x})\right)}$$

Trong phương trình này:

- *K* là số lớp.
- $\mathbf{s}(\mathbf{x})$ là một vectơ chứa các điểm số của từng lớp đối với mẫu **x**.
- $\sigma(\mathbf{s}(\mathbf{x}))_k$là xác suất ước tính rằng mẫu **x** thuộc lớp *k*, dựa trên các điểm số của từng lớp đối với mẫu đó.

Cũng giống như bộ phân loại hồi quy logistic, theo mặc định, bộ phân loại hồi quy softmax dự đoán lớp có xác suất ước tính cao nhất (đơn giản là lớp có điểm số cao nhất), như được thể hiện trong Phương trình 4-22.

Phương trình 4-22. Dự đoán của mô hình hồi quy Softmax

$$\hat{y} = \underset{k}{\operatorname{argmax}}\ \sigma(s(x))_k = \underset{k}{\operatorname{argmax}}\ s_k(x) = \underset{k}{\operatorname{argmax}}\ \left(\left(\theta^{(k)}\right)^T x\right)$$

Toán tử *argmax* trả về giá trị của một biến làm tối đa hóa một hàm. Trong phương trình này, nó trả về giá trị của k làm tối đa hóa xác suất ước lượng $\sigma(\mathbf{s}(\mathbf{x}))_k$.



Bộ phân loại hồi quy softmax chỉ dự đoán một lớp tại một thời điểm (tức là, nó là đa lớp, không phải đa đầu ra), vì vậy nó chỉ nên được sử dụng với các lớp loại trừ lẫn nhau, chẳng hạn như các loài thực vật khác nhau. Bạn không thể sử dụng nó để nhận diện nhiều người trong một bức ảnh.

Giờ đây, khi bạn đã biết cách mô hình ước lượng xác suất và đưa ra dự đoán, hãy cùng tìm hiểu về quá trình huấn luyện. Mục tiêu là xây dựng một mô hình ước lượng xác suất cao cho lớp mục tiêu (và do đó, xác suất thấp cho các lớp còn lại). Việc tối thiểu hóa hàm chi phí được thể hiện trong Phương trình 4-23, được gọi là *entropy chéo*, sẽ dẫn đến mục tiêu này vì nó phạt mô hình khi mô hình ước tính xác suất thấp cho lớp mục tiêu. Entropy chéo thường được sử dụng để đo lường mức độ phù hợp giữa tập hợp các xác suất lớp được ước tính với các lớp mục tiêu.

Phương trình 4-23. Hàm chi phí entropy chéo

$$J(\Theta) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K} y_k^{(i)} \log\left(\hat{p}_k^{(i)}\right)$$

Trong phương trình này, $y_k^{(i)}$ là xác suất mục tiêu cho rằng mẫu thứ i thuộc lớp k. Nói chung, giá trị này bằng 1 hoặc 0, tùy thuộc vào việc mẫu đó có thuộc lớp hay không.

Lưu ý rằng khi chỉ có hai lớp ($K = 2$), hàm chi phí này tương đương với hàm chi phí hồi quy logistic (log loss; xem Phương trình 4-18).

Entropy chéo

Entropy chéo có nguồn gốc từ *lý thuyết thông tin* của Claude Shannon. Giả sử bạn muốn truyền tải hiệu quả thông tin về thời tiết mỗi ngày. Nếu có tám tùy chọn (nắng, mưa, v.v.), bạn có thể mã hóa mỗi tùy chọn bằng 3 bit, vì $2^3 =$ 8. Tuy nhiên, nếu bạn cho rằng hầu như ngày nào cũng nắng, sẽ hiệu quả hơn nhiều nếu mã hóa "nắng" chỉ bằng một bit (0) và bảy tùy chọn còn lại bằng bốn bit (bắt đầu bằng số 1). Entropy chéo đo lường số bit trung bình mà bạn thực sự gửi cho mỗi tùy chọn. Nếu giả định của bạn về thời tiết là hoàn hảo, entropy chéo sẽ bằng entropy của chính thời tiết (tức là tính không thể dự đoán nội tại của nó). Nhưng nếu

giả định của bạn sai (ví dụ: nếu trời thường xuyên mưa), entropy chéo sẽ lớn hơn một mức độ được gọi là *độ lệch Kullback–Leibler (KL)*.

Entropy chéo giữa hai phân phối xác suất p và q được định nghĩa là $H(p,q) = -\Sigma_x p(x) \log q(x)$ (ít nhất là khi các phân phối là rời rạc). Để biết thêm chi tiết, hãy xem video của tôi về chủ đề này.

Vectơ gradient của hàm chi phí này đối với $\boldsymbol{\theta}^{(k)}$ được cho bởi Phương trình 4-24.

Phương trình 4-24. Vectơ gradient entropy chéo cho lớp k

$$\nabla_{\boldsymbol{\theta}^{(k)}} J(\boldsymbol{\Theta}) = \frac{1}{m} \sum_{i=1}^{m} \left(\hat{p}_k^{(i)} - y_k^{(i)} \right) \mathbf{x}^{(i)}$$

Bây giờ bạn có thể tính toán vectơ gradient cho mỗi lớp, sau đó sử dụng phương pháp gradient descent (hoặc bất kỳ thuật toán tối ưu hóa nào khác) để tìm ma trận tham số $\boldsymbol{\Theta}$ làm tối thiểu hàm chi phí.

Hãy sử dụng hồi quy softmax để phân loại các loài hoa diên vĩ thành cả ba lớp. Bộ phân loại `LogisticRegression` của Scikit-Learn sẽ tự động sử dụng hồi quy softmax khi bạn huấn luyện nó trên hơn hai lớp (giả sử bạn sử dụng `solver="lbfgs"`, đây là giá trị mặc định). Nó cũng áp dụng chuẩn hóa ℓ_2 theo mặc định, mà bạn có thể điều khiển bằng siêu tham số `C`: giảm `C` để tăng mức độ chuẩn hóa, như đã đề cập trước đó.

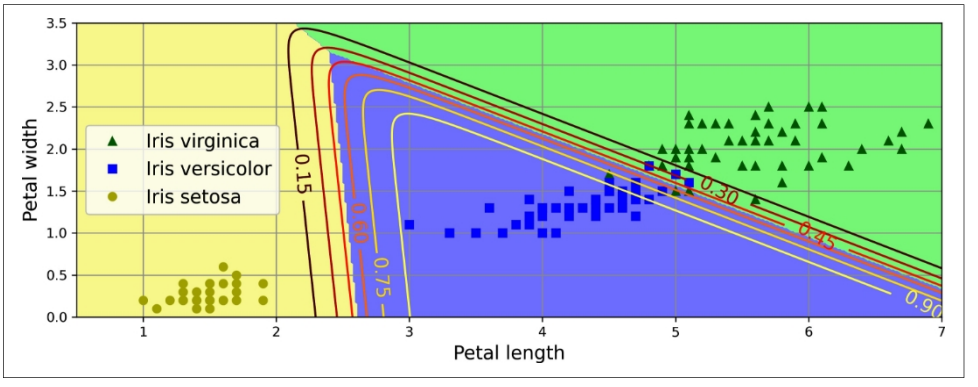
```
X = iris.data[["chiều dài cánh hoa (cm)", "chiều rộng cánh hoa
(cm)"]].values y = iris["target"]
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

softmax_reg = LogisticRegression(C=30, random_state=42)
softmax_reg.fit(X_train, y_train)
```

Vì vậy, lần tới khi bạn tìm thấy một bông hoa diên vĩ có cánh hoa dài 5 cm và rộng 2 cm, bạn có thể yêu cầu mô hình của mình cho biết đó là loại hoa diên vĩ nào, và mô hình sẽ trả lời *là Iris virginica* (loại 2) với xác suất 96% (hoặc *Iris versicolor* với xác suất 4%):

```
>>> softmax_reg.predict([[5, 2]])
array([2])
>>> softmax_reg.predict_proba([[5, 2]]).round(2)
array([[0. , 0.04, 0.96]])
```

Hình 4-26 cho thấy các ranh giới quyết định thu được, được biểu diễn bằng các màu nền. Lưu ý rằng các ranh giới quyết định giữa bất kỳ hai lớp nào đều là tuyến tính. Hình này cũng cho thấy các xác suất cho lớp *Iris versicolor*, được biểu diễn bằng các đường cong (ví dụ: đường có nhãn 0,30 biểu diễn ranh giới xác suất 30%). Lưu ý rằng mô hình có thể dự đoán một lớp có xác suất ước tính dưới 50%. Ví dụ, tại điểm mà tất cả các đường ranh giới quyết định giao nhau, tất cả các lớp đều có xác suất ước tính bằng nhau là 33%.



Hình 4-26. Các đường ranh giới quyết định trong hồi quy Softmax

Trong chương này, bạn đã tìm hiểu các phương pháp khác nhau để huấn luyện mô hình tuyến tính, cả cho hồi quy lẫn phân loại. Bạn đã sử dụng phương trình dạng đóng để giải bài toán hồi quy tuyến tính, cũng như phương pháp gradient descent, và đã tìm hiểu cách thêm các hàm phạt vào hàm chi phí trong quá trình huấn luyện để thường quy hóa mô hình. Trong quá trình đó, bạn cũng đã học cách vẽ và phân tích các đường cong học tập, cũng như cách triển khai cơ chế dừng sớm. Cuối cùng, bạn đã tìm hiểu cách hoạt động của hồi quy logistic và hồi quy Softmax. Chúng ta đã mở ra những "hộp đen" đầu tiên của học máy! Trong các chương tiếp theo, chúng ta sẽ mở ra nhiều "hộp đen" hơn nữa, bắt đầu với máy vectơ hỗ trợ (Support Vector Machines).

Bài tập

1. Bạn có thể sử dụng thuật toán huấn luyện hồi quy tuyến tính nào nếu có tập huấn luyện với hàng triệu đặc trưng?
2. Giả sử các đặc trưng trong tập dữ liệu huấn luyện của bạn có các thang đo rất khác nhau. Những thuật toán nào có thể bị ảnh hưởng bởi điều này, và như thế nào? Bạn có thể làm gì để khắc phục điều này?
3. Phương pháp gradient descent có thể bị kẹt ở điểm cực tiểu cục bộ khi huấn luyện mô hình hồi quy logistic không?
4. Tất cả các thuật toán gradient descent có dẫn đến cùng một mô hình không, miễn là bạn để chúng chạy đủ lâu?
5. Giả sử bạn sử dụng thuật toán gradient descent theo lô và vẽ biểu đồ sai số xác nhận ở mỗi epoch. Nếu bạn nhận thấy sai số xác nhận liên tục tăng lên, điều gì có thể đang xảy ra? Làm thế nào để khắc phục vấn đề này?
6. Có nên dừng ngay thuật toán gradient descent theo lô nhỏ khi sai số xác nhận tăng lên không?
7. Trong số các thuật toán gradient descent mà chúng ta đã thảo luận, thuật toán nào sẽ đạt đến vùng lân cận của giải pháp tối ưu nhanh nhất? Thuật toán nào thực sự sẽ hội tụ? Làm thế nào để các thuật toán khác cũng hội tụ được?

8. Giả sử bạn đang sử dụng hồi quy đa thức. Bạn vẽ biểu đồ các đường cong học tập và nhận thấy có một khoảng cách lớn giữa sai số huấn luyện và sai số xác nhận. Điều gì đang xảy ra? Có ba cách nào để giải quyết vấn đề này?
9. Giả sử bạn đang sử dụng hồi quy ridge và nhận thấy rằng sai số huấn luyện và sai số xác nhận gần như bằng nhau và khá cao. Bạn có cho rằng mô hình này bị sai lệch cao hay phương sai cao không? Bạn nên tăng tham số siêu α hay giảm nó?
10. Tại sao bạn lại muốn sử dụng:
    a. hồi quy Ridge thay vì hồi quy tuyến tính đơn thuần (tức là không có bất kỳ quá trình thường quy nào)?
    b. Phương pháp Lasso thay vì hồi quy Ridge?
    c. Hồi quy Elastic Net thay vì hồi quy Lasso?
11. Giả sử bạn muốn phân loại các hình ảnh thành ngoài trời/trong nhà và ban ngày/ban đêm. Bạn nên triển khai hai bộ phân loại hồi quy logistic hay một bộ phân loại hồi quy softmax?
12. Thực hiện thuật toán gradient descent theo lô với cơ chế dừng sớm cho hồi quy softmax mà không sử dụng Scikit-Learn, chỉ dùng NumPy. Áp dụng nó cho một tác vụ phân loại như tập dữ liệu Iris.

Các giải pháp cho các bài tập này có sẵn ở cuối sổ tay của chương này, tại *https://homl.info/colab-p*.